

UVSQ - UNIVERSITÉ PARIS SACLAY

ISTY - Institut des Sciences et Techniques des Yvelines

MÉMOIRE DE FIN D'ÉTUDES

Master en Ingénierie des Systèmes et Réseaux

**Conception et mise en œuvre d'une infrastructure
d'intégration et de livraison continues CI/CD pour
le firmware de communication d'un compteur
d'énergie industriel (Power Meter)**

Présenté par :

Abdillahi Mohamed

Encadrant académique :

Mourad GUEROUI

Encadrant entreprise :

Matthieu RIEUX, Schneider Electric

Clause de confidentialité

Le présent mémoire a été réalisé dans le cadre d'un projet industriel mené au sein de Schneider Electric. Il décrit une infrastructure interne d'intégration et de livraison continues ainsi que les outils et processus associés.

Certaines informations (dénominations commerciales, identifiants de projets, valeurs de mesures) ont été anonymisées ou omises afin de préserver la confidentialité des données de l'entreprise. Les éléments techniques présentés le sont à des fins exclusivement pédagogiques et d'évaluation académique.

La diffusion de ce document est restreinte aux membres du jury et aux personnels de l'établissement habilités, sauf autorisation écrite contraire de Schneider Electric.

Remerciements

Je tiens à remercier Matthieu RIEUX, mon encadrant au sein de Schneider Electric, pour la confiance accordée et la belle dose d'autonomie laissée sur ce projet, ainsi que pour le temps pris à me former, aussi bien sur les aspects techniques que professionnels — un accompagnement qui a fait toute la différence.

Mes remerciements vont également à l'ensemble de l'équipe COM, en particulier Fabrice, Silvia et Yannick, pour leur soutien tout au long de ce projet.

Je remercie enfin Mourad GUEROUI, mon encadrant académique, qui a su garder un œil bienveillant (et patient) sur l'avancement de ce mémoire, ainsi que les membres du jury qui ont accepté d'évaluer ce travail.

Merci à mes amis et à mes proches pour leur soutien tout au long de ce travail.

Résumé

Le développement du firmware des produits industriels embarqués se heurte à des cycles d'intégration longs, à des environnements de compilation hétérogènes et à des exigences de cybersécurité croissantes. Ce mémoire présente la conception et la mise en œuvre d'une infrastructure d'intégration et de livraison continues pour le firmware du module de communication d'un compteur d'énergie (*Power Meter*) développé chez Schneider Electric, module chargé d'intégrer les protocoles Modbus, Ethernet, BACnet et une API REST.

Le travail ne part pas d'une feuille blanche : il reprend une chaîne GitHub Actions existante, en analyse les limites, formalise des exigences avec l'équipe, puis la fait évoluer. Les évolutions portent sur une image de compilation unique intégrant le kit de développement de la cible, une matrice de construction native et croisée, la mesure de couverture, les tests d'intégration, l'analyse de composition binaire, la publication des artefacts et la collecte d'indicateurs vers VictoriaMetrics et Grafana.

L'évaluation porte sur 164 exécutions sur cinq semaines. La chaîne applicative rend un résultat complet en 21,8 minutes en médiane, la parallélisation de la matrice réduisant sa durée d'environ un tiers. La chaîne restitue à chaque exécution la couverture (32,3 % des lignes), les défauts d'analyse statique et l'inventaire des composants tiers avec leurs vulnérabilités. La distribution Linux reste reconstruite sans cache en 68,2 minutes, ce qui fait du cache partagé la principale perspective.

Mots-clés : intégration continue, livraison continue, DevOps, systèmes embarqués, Yocto, analyse statique, chaîne d'approvisionnement logicielle, analyse de composition, observabilité.

Abstract

The development of firmware for embedded industrial products is hindered by long integration cycles, heterogeneous build environments, and growing cybersecurity requirements. This thesis presents the design and implementation of a continuous integration and continuous delivery infrastructure for the firmware of the communication module of an energy meter (*Power Meter*) developed at Schneider Electric, a module responsible for integrating the Modbus, Ethernet, BACnet, and REST API protocols.

Rather than starting from scratch, the work takes over an existing GitHub Actions pipeline, analyses its limits, formalises requirements with the team and evolves it : a single build image embedding the target SDK, a native and cross-compilation build matrix, code coverage, integration tests, binary composition analysis, artifact publication, and metrics collection into VictoriaMetrics and Grafana.

The evaluation covers 164 runs over five weeks. The application pipeline delivers a complete result in a median of 21.8 minutes, the parallel matrix cutting its duration by about one third. Each run reports coverage (32.3% of lines), static analysis defects and the inventory of third-party components with their vulnerabilities. The Linux distribution is still rebuilt without cache in 68.2 minutes, which makes a shared build cache the main next step.

Keywords : continuous integration, continuous delivery, DevOps, embedded systems, Yocto, static analysis, software supply chain, software composition analysis, observability.

Table des matières

Remerciements	iii
Résumé	iv
1 Introduction	1
1.1 Contexte général	1
1.2 Problématique	1
1.3 Objectifs et périmètre	2
1.4 Contributions	2
1.5 Organisation du mémoire	2
2 Contexte industriel	3
2.1 Schneider Electric	3
2.2 Le projet Bluebird	4
2.2.1 Présentation et objectifs	4
2.2.2 Organisation des équipes	5
2.3 Mon rôle	6
2.3.1 Missions et périmètre	6
2.4 Synthèse	6
3 CI/CD pour les systèmes embarqués : état de l’art	7
3.1 L’intégration continue	7
3.2 Qualité et sécurité logicielle	8
3.3 L’infrastructure d’exécution	8
3.4 L’observabilité de la chaîne	9
3.5 Synthèse et positionnement	9
4 Analyse des besoins, conception et mise en œuvre des évolutions de la chaîne CI/CD	11
4.1 Analyse de l’existant et recueil des besoins	11
4.1.1 Situation à mon arrivée	11
4.1.2 Méthode de recueil	11
4.1.3 Besoins identifiés	11
4.2 Formalisation des exigences	12
4.2.1 Parties prenantes	12

4.2.2	Exigences fonctionnelles	12
4.2.3	Exigences non fonctionnelles	13
4.2.4	Contraintes et hypothèses	14
4.3	Conception des évolutions de la chaîne CI/CD	15
4.3.1	Principes directeurs	15
4.3.2	Contraintes imposées et décisions d'intégration	16
4.3.3	Architecture cible	18
4.3.4	Décomposition en chaînes	19
4.4	Mise en œuvre technique	20
4.4.1	Les chaînes CI	20
4.4.2	L'infrastructure d'exécution	31
4.4.3	L'observabilité	33
4.4.4	Bilan	34
5	Évaluation et résultats	37
5.1	Protocole d'évaluation	37
5.1.1	Questions d'évaluation	37
5.1.2	Sources de mesure	37
5.1.3	Validité	38
5.2	Impact	38
5.2.1	Indicateurs de performance de livraison	39
5.2.2	Coût et effort d'administration	39
5.2.3	Retour des utilisateurs	40
5.2.4	Apports pour l'entreprise	40
5.3	Difficultés rencontrées	40
5.3.1	Difficultés techniques	40
5.3.2	Difficultés liées à la mesure	40
5.4	Résultats	41
5.4.1	Reproductibilité de l'environnement	41
5.4.2	Performance de la chaîne	41
5.4.3	Qualité et sécurité	42
5.5	Discussion	43
5.5.1	Réponses aux questions d'évaluation	43
5.5.2	Limites	43
5.5.3	Menaces à la validité	44
6	Conclusion et perspectives	45
6.1	Rappel de la problématique et de la démarche	45
6.2	Principaux résultats	45
6.3	Apports personnels	46
6.3.1	Montée en compétences techniques	46
6.3.2	Intégration dans une organisation industrielle	46
6.3.3	Projet professionnel	47

6.4	Perspectives	47
6.4.1	Court terme	47
6.4.2	Moyen terme	47
6.4.3	Long terme	48
6.5	Conclusion	48
Bibliographie		49
Données de mesure		51
.1	Relevé des exécutions	51
.2	Mesures de couverture	51
.3	Relevé des défauts d'analyse statique	52
.4	Relevé des vulnérabilités de composants	52

Table des figures

2.1	Schneider Electric dans le monde — chiffres clés 2024	3
2.2	Organisation du groupe Schneider Electric	4
2.3	Centrale de mesure PowerLogic PM77xx (sous-vague 1.2)	5
4.1	Architecture générale de la chaîne d'intégration et de livraison	18
4.2	Historique des exécutions de pm77xx-app depuis la création du workflow (1 186 exécutions)	20
4.3	Exécution de pm77xx-app (#1316) : matrice de construction et jobs aval . . .	22
4.4	Synthèse d'exécution : défauts Coverity, tests, couverture et BDBA	24
4.5	Rapport BDBA de l'archive pm77xx-app (RELEASE/aarch64)	27
4.6	Exécution de pm77xx-yocto (#543, 1 h 05 min 27 s)	28
4.7	Organisation du dépôt générique Artifactory	29
4.8	Exécution de publish-image : construction, contrôles, validation, publication	31
4.9	Tableau de bord Grafana de l'équipe COM	34

Liste des tableaux

3.1	Positionnement du travail par rapport à l'état de l'art	9
4.1	Parties prenantes et attentes principales	12
4.2	Exigences fonctionnelles	13
4.3	Exigences non fonctionnelles	14
4.4	Éléments imposés et décisions d'intégration	17
4.5	Situation à mon arrivée et évolutions apportées	19
4.6	Chaînes d'intégration	19
4.7	Paramètres d'entrée de pm77xx-app	21
4.8	Répartition des activités sur la matrice	21
4.9	Indicateurs extraits de l'analyse de composition	26
4.10	Variante d'image produites	28
4.11	Principaux composants de l'image et versions épinglées	30
4.12	Solutions étudiées pour le cache de la distribution	32
4.13	Indicateurs collectés vers VictoriaMetrics	33
4.14	Couverture des exigences fonctionnelles	35
5.1	Sources des mesures utilisées	37
5.2	Indicateurs de performance adaptés au contexte embarqué	39
5.3	Durées de construction observées (minutes)	41
6.1	Contributions et résultats	46
2	Synthèse des exécutions par chaîne (11 août – 17 septembre 2026)	51
3	Extrait du relevé des exécutions	51

Liste des acronymes

COM Module Communication (interfaces Modbus, BACnet, Ethernet, API REST)

CVE Common Vulnerabilities and Exposures

CVSS Common Vulnerability Scoring System

DORA DevOps Research and Assessment

MET Module Metering (acquisition et mesure des grandeurs électriques)

OMO Offer Marketing Owner

OTL Offer Technical Leader

SBOM Software Bill of Materials

SDK Software Development Kit

SoC System on Chip

Chapitre 1

Introduction

1.1 Contexte général

Bien que l'intégration continue et la livraison continue (CI/CD) soient devenues des pratiques essentielles dans le développement logiciel moderne, leur application au développement de firmware embarqué industriel reste complexe en raison de contraintes spécifiques telles que les temps de compilation élevés, la dépendance au matériel, l'utilisation d'outils propriétaires et l'impossibilité de déployer continuellement les mises à jour sur les produits installés. À ces défis s'ajoutent des exigences croissantes de cybersécurité, notamment en matière de traçabilité des composants, de détection des vulnérabilités et de démonstration d'un cycle de développement sécurisé. L'adaptation des principes CI/CD à ce contexte constitue donc un véritable problème d'ingénierie nécessitant une approche dédiée, que ce mémoire étudie à travers un cas industriel concret.

1.2 Problématique

Le développement du firmware d'un produit industriel embarqué cumule des contraintes que l'on rencontre rarement simultanément dans le développement logiciel classique :

- des temps de construction très longs, la génération d'une distribution Linux embarquée complète se comptant en heures et non en minutes ;
- un environnement de compilation complexe et fragile, associant chaînes de compilation croisée, bibliothèques système et outils propriétaires ;
- des exigences réglementaires de cybersécurité imposant la traçabilité des composants et la détection précoce des vulnérabilités ;
- une exécution des traitements sur une infrastructure éphémère, où aucun état ne persiste d'une exécution à l'autre.

Ces contraintes conduisent à la question de recherche suivante :

Comment concevoir et mettre en œuvre une chaîne d'intégration et de livraison continues reproductible, sécurisée et mesurable pour le firmware d'un produit industriel embarqué, sous contraintes de temps de construction longs, d'exigences de cybersécurité et d'infrastructure d'exécution éphémère ?

1.3 Objectifs et périmètre

L'objectif est de faire évoluer une chaîne CI/CD existante pour qu'elle soit reproductible (un environnement de compilation unique et versionné), sécurisée (analyse statique et analyse de composition intégrées à chaque exécution) et mesurable (indicateurs collectés, historisés et visualisés).

Le périmètre de ce travail est la *chaîne d'intégration* du firmware, et non le firmware lui-même. Sont explicitement exclus : la conception fonctionnelle de l'application embarquée, ainsi que la conception matérielle du produit.

1.4 Contributions

Les contributions de ce travail sont les suivantes :

1. la conteneurisation de l'environnement de compilation applicatif ;
2. la consolidation de deux chaînes d'intégration distinctes et complémentaires, l'une pour la couche applicative et l'autre pour la distribution Linux embarquée ;
3. l'intégration automatisée de l'analyse statique et de l'analyse de composition logicielle avec restitution des résultats au développeur ;
4. l'analyse des contraintes des exécuteurs éphémères et l'étude de solutions de cache partagé (NFS, snapshot) ;
5. la mise en place d'une solution d'observabilité restituant l'évolution des indicateurs de qualité dans le temps.

1.5 Organisation du mémoire

Le chapitre 2 présente le contexte industriel et ma mission. Le chapitre 3 résume l'état de l'art sur l'intégration continue appliquée à l'embarqué. Le chapitre 4 analyse l'existant, formalise les exigences et décrit la conception et la mise en œuvre des évolutions. Le chapitre 5 évalue la chaîne obtenue à partir des exécutions réelles, et le chapitre 6 conclut et ouvre des perspectives.

Chapitre 2

Contexte industriel

2.1 Schneider Electric

Schneider Electric est un groupe industriel français spécialisé dans la gestion de l'énergie et l'automatisation. Le groupe conçoit des équipements et des logiciels permettant de produire, distribuer, mesurer et piloter l'énergie électrique, pour des clients allant du bâtiment tertiaire et résidentiel à l'industrie lourde, en passant par les infrastructures critiques et les data centers. Il est implanté dans de nombreux pays et organise son activité en plusieurs grandes divisions, correspondant chacune à un segment de marché (bâtiment, industrie, infrastructures, énergie).

Le groupe réalise, en 2024, un chiffre d'affaires de 38,2 milliards d'euros et emploie plus de 160 000 collaborateurs dans plus de 100 pays. Il consacre 6 % de son chiffre d'affaires à la recherche et au développement et se classe 5^e capitalisation du CAC 40. Son activité se structure autour de deux cœurs de métier complémentaires, la gestion de l'énergie et les automatismes industriels, couvrant l'ensemble de la chaîne de valeur, de la source d'énergie jusqu'à la ligne de production. Ses marchés finaux se répartissent entre les bâtiments (30 %), l'industrie (33 %), les centres de données (24 %) et les infrastructures (13 %) (commandes 2024).

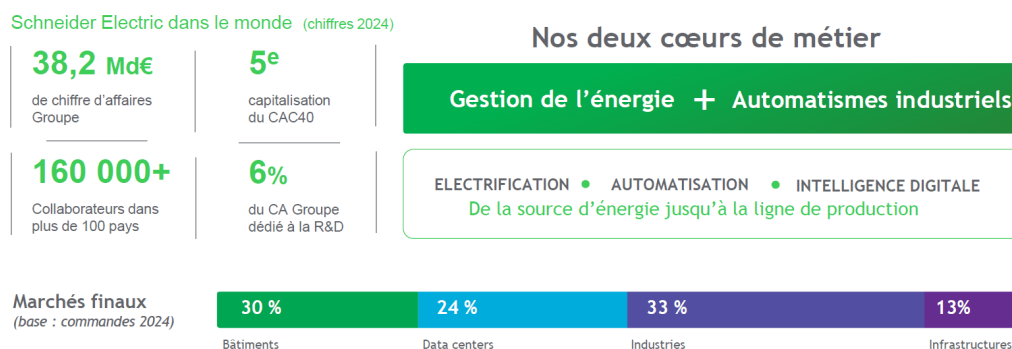


Figure 2.1 – Schneider Electric dans le monde — chiffres clés 2024

L'organisation du groupe repose sur deux *business units* correspondant à ces deux cœurs de métier (*Energy Management* et *Industrial Automation*), qui conçoivent les offres, produits

et services. Les opérations, qui conseillent les clients et commercialisent ces offres, sont organisées en quatre régions : Europe, Amérique du Nord, Chine et International Operations. La fabrication et l’acheminement des produits sont assurés par une chaîne d’approvisionnement mondiale (*Global Supply Chain*), forte d’environ 200 usines et 100 centres de distribution. Des fonctions globales — finance, marketing, Schneider Digital notamment — apportent leur support à l’ensemble de ces entités.

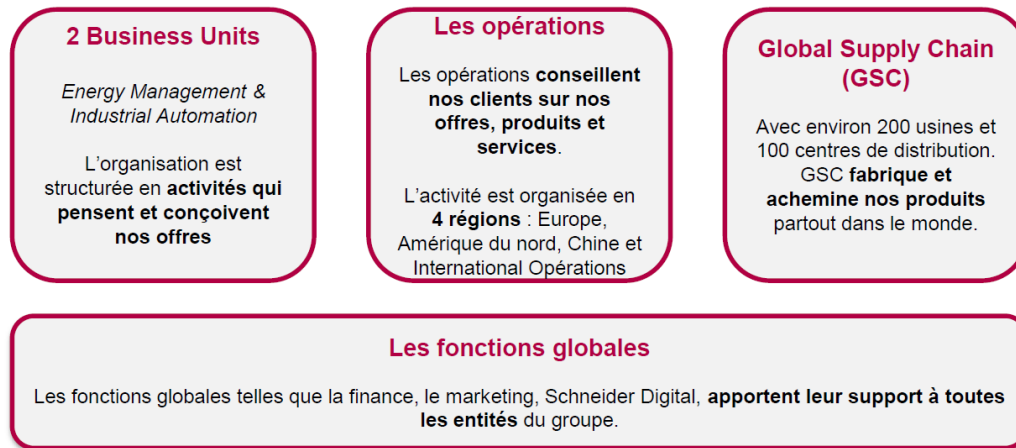


Figure 2.2 – Organisation du groupe Schneider Electric

Mon stage s’est déroulé au sein de la *business unit* Energy Management, dans la division *Digital Energy*, qui porte le programme Bluebird et le développement du produit *Power Meter*.

2.2 Le projet Bluebird

2.2.1 Présentation et objectifs

Bluebird est le programme de renouvellement de la gamme de comptage d’énergie standalone de Schneider Electric, porté par la division *Digital Energy* sous la marque PowerLogic. Il répond à un contexte réglementaire et énergétique porteur : la directive européenne sur la performance énergétique des bâtiments, volet du *Green Deal*, impose progressivement la mise en place de systèmes de gestion technique du bâtiment et le comptage continu des consommations, y compris pour des puissances de chauffage/climatisation de plus en plus faibles à mesure que les seuils réglementaires sont abaissés. Le programme vise à inventer une troisième génération de comptage : instrumenter chaque tableau électrique par conception et à coût maîtrisé, tout en conservant le leadership de Schneider Electric sur le marché du comptage standalone.

Le programme est structuré en plusieurs vagues (*waves*) successives. La vague 1 renouvelle l’offre historique (gamme PM5000) et introduit un véritable *système* de comptage modulaire : des compteurs standalone (sous-vague 1.1, familles PM71/72/73) et, surtout, une nouvelle génération de centrales de mesure formant un serveur d’énergie modulaire (sous-vague 1.2, familles PM77/78/79), capable d’agréger plusieurs capteurs *PowerTag* et de communiquer simultanément par plusieurs canaux filaires et sans fil (Ethernet Modbus/BACnet, Wi-Fi,

Zigbee, Bluetooth). Les vagues suivantes étendent le concept aux capteurs de courant intelligents (vague 2) puis aux capteurs d'énergie auto-alimentés destinés à la rénovation de tableaux existants (vague 3).



Figure 2.3 – Centrale de mesure PowerLogic PM77xx (sous-vague 1.2)

Sur le plan technique, le PM77xx repose sur un System on Chip (SoC) Texas Instruments AM642 (cœurs Linux et temps réel) et embarque un firmware composé d'une distribution Linux (Yocto), d'une brique de cybersécurité transverse et des modules applicatifs MET et COM décrits ci-dessous.

2.2.2 Organisation des équipes

Le projet Bluebird est porté par plusieurs équipes pluridisciplinaires, organisées autour des grands blocs du produit présentés ci-dessus.

Management

Le pilotage du programme Bluebird est assuré par une équipe Management distincte des équipes de développement : un *Scrum Master* anime le cadre agile et la synchronisation des équipes HW et SW, des *Business Developers* portent la relation avec les marchés et les clients cibles, et un Offer Technical Leader (OTL) ainsi qu'un Offer Marketing Owner (OMO) pilotent respectivement la coordination transverse de l'offre et son positionnement marketing.

Hardware (HW)

L'équipe *Hardware* (HW) conçoit et qualifie la partie matérielle du PM77xx : carte porteuse, choix et intégration du SoC Texas Instruments AM642, qualification électrique et environnementale. Elle fournit à l'équipe logicielle les spécifications matérielles nécessaires à la compilation croisée et à la génération du firmware temps réel.

Software (SW)

Le firmware applicatif du PM77xx est développé par deux équipes distinctes, correspondant aux deux modules fonctionnels du produit.

Équipe MET. L'équipe Module Metering (acquisition et mesure des grandeurs électriques) (MET) développe le module de mesure, premier à être réalisé car il constitue le socle métier du produit : il assure l'acquisition, le calcul et la journalisation des grandeurs électriques mesurées. Sans acquisition ni calcul fiable de ces grandeurs, aucune donnée ne peut être exposée.

Équipe COM. L'équipe Module Communication (interfaces Modbus, BACnet, Ethernet, API REST) (COM) développe le module de communication, en s'appuyant sur les données produites par le MET : il expose ces grandeurs et les rend accessibles aux systèmes tiers, à travers les interfaces Modbus, BACnet, Ethernet et une API REST. C'est au sein de cette équipe que s'est déroulée ma mission, et c'est l'industrialisation de la chaîne de construction et de livraison de ce module qui constitue le périmètre de ce mémoire.

2.3 Mon rôle

2.3.1 Missions et périmètre

J'ai rejoint l'équipe COM en tant que stagiaire DevSecOps, sur le périmètre de la chaîne d'intégration et de livraison continues du module de communication. Mon rôle ne portait donc pas sur le développement fonctionnel du firmware lui-même, mais sur l'outillage qui permet de le construire, de le vérifier et de le livrer de manière automatisée et reproductible.

Mes missions correspondent aux cinq contributions présentées en section 1.4 : environnement de compilation conteneurisé, chaînes applicative et de distribution, analyses de sécurité, contraintes de l'infrastructure d'exécution et observabilité.

Cette mission s'est déroulée sur 6 mois, au sein d'une phase du programme Bluebird où la vague 1.2 (PM77xx) était déjà en développement actif : la chaîne d'intégration existait donc avant mon arrivée, et ma mission a consisté à en prendre la responsabilité, à en corriger les défauts et à la faire évoluer, plutôt qu'à la concevoir depuis une feuille blanche. Le développement fonctionnel du firmware restait à la charge des développeurs de l'équipe COM ; mon périmètre se limitait à l'outillage de construction, de vérification et de livraison décrit ci-dessus.

2.4 Synthèse

Ce chapitre a présenté le cadre industriel de ce mémoire : Schneider Electric, le programme Bluebird et le PM77xx, ainsi que le périmètre concret de ma mission au sein de l'équipe COM. Avant de détailler les décisions de conception et leur mise en œuvre, il convient d'examiner ce que la littérature et l'industrie proposent déjà pour les problèmes rencontrés sur ce type de chaîne — c'est l'objet du chapitre suivant.

Chapitre 3

CI/CD pour les systèmes embarqués : état de l’art

Ce chapitre présente, dans l’ordre où ils apparaissent dans une chaîne, quatre piliers : l’intégration continue elle-même, la qualité et la sécurité qui s’y greffent, l’infrastructure qui l’exécute, et l’observabilité qui permet de la piloter. Chaque section se termine par ce que l’on en retient pour ce travail.

3.1 L’intégration continue

L’intégration continue consiste à intégrer fréquemment le travail de chaque développeur dans une ligne principale partagée, chaque intégration étant validée par une construction et des tests automatisés [4, 2]. La livraison continue maintient en outre le logiciel dans un état déployable [6]. Les études empiriques confirment le bénéfice de cette automatisation sur la qualité et la productivité [18], mais aussi son coût de maintenance [5] et la forte variabilité des mises en œuvre industrielles [17].

Le contexte embarqué complique cet enchaînement. Le livrable est une image complète (système, composants de base, services applicatifs) et non un simple binaire ; la chaîne doit donc produire des artefacts de rythmes très différents, l’appliquatif d’un côté et l’image bas niveau, dont la construction se compte en heures, de l’autre. Les retours industriels recensent la compilation croisée, la dépendance au matériel pour une partie des tests et la durée des constructions [11, 16], et soulignent que les pratiques DevOps s’adaptent mal aux produits dont le déploiement n’est pas continu [10]. La *reproductibilité* du livrable — obtenir le même artefact à partir de la même révision, quelles que soient la machine et la date — est de plus un levier d’intégrité de la chaîne d’approvisionnement logicielle [9].

Ce que l’on retient. Ces contraintes justifient de séparer la chaîne applicative de celle de l’image bas niveau, et de traiter la reproductibilité de l’environnement de compilation comme un prérequis.

3.2 Qualité et sécurité logicielle

Une chaîne qui compile et teste garantit que le logiciel fonctionne, non qu'il est de bonne qualité ni sûr. La mesure de couverture indique la part du code exercée par les tests (lignes, fonctions, branches) ; elle s'interprète par sa *tendance* plus que par sa valeur absolue.

L'analyse statique recherche des défauts sans exécuter le code. Elle est largement adoptée mais souvent configurée à minima et peu intégrée au flux quotidien [1] ; la difficulté est organisationnelle plus que technique, le volume de signalements et les faux positifs conditionnant son adoption [19]. Introduite sur une base existante, elle révèle une dette que l'on ne résorbe pas à court terme : on peut soit faire échouer la construction dès un défaut, soit se limiter à notifier. Ce travail retient la notification, justifiée au chapitre 4 et discutée au chapitre 5.

L'analyse de composition répond au fait qu'un produit embarqué agrège de nombreux composants tiers, chacun pouvant porter des vulnérabilités ; les attaques sur la chaîne d'approvisionnement sont devenues un mode de compromission à part entière [13]. Elle s'appuie sur la nomenclature logicielle (Software Bill of Materials (SBOM)), dont les éléments minimaux ont été définis par la NTIA [12] et le format SPDX normalisé par l'ISO/IEC 5962 [8], puis sur la corrélation aux vulnérabilités publiques (Common Vulnerabilities and Exposures (CVE)), hiérarchisées par le score Common Vulnerability Scoring System (CVSS) et classées selon les catégories de l'OWASP [14]. Dans une distribution embarquée, analyser l'artefact compilé plutôt que les seuls fichiers de dépendances permet d'identifier les composants réellement présents dans l'image.

Ces pratiques sont désormais attendues par le cadre normatif : la norme IEC 62443-4-1 [7] définit les exigences de développement sécurisé pour l'automatisation industrielle, et le règlement (UE) 2024/2847 [15] impose notamment la tenue d'une nomenclature logicielle et le traitement des vulnérabilités pendant toute la durée de support du produit.

Ce que l'on retient. Ces vérifications n'ont de valeur que si elles sont automatiques, systématiques et restituées au développeur au moment où il peut agir.

3.3 L'infrastructure d'exécution

Deux modèles coexistent : des machines permanentes, réutilisées d'une exécution à l'autre, ou des exécuteurs *éphémères*, alloués puis détruits à chaque exécution. Le second garantit que deux exécutions partent de conditions identiques, mais interdit toute persistance locale, alors que les constructions embarquées reposent sur la réutilisation de résultats intermédiaires. La conteneurisation complète ce dispositif : décrire l'environnement de compilation sous forme d'image versionnée donne une base unique pour la chaîne et pour les postes des développeurs, et supprime une cause classique de divergence entre résultats locaux et résultats d'intégration.

Ce que l'on retient. Les exécuteurs éphémères sont une contrainte d'entreprise, non un choix d'architecture ; avec la conteneurisation, ils structurent l'architecture décrite au chapitre 4.

3.4 L'observabilité de la chaîne

Une exécution ne produit qu'un résultat ponctuel (réussite ou échec) qui ne dit rien de l'évolution du produit ni de la chaîne. Piloter la qualité suppose de *collecter* les indicateurs, de les *traduire* vers un format commun historisable, puis de les *formaliser* sous une forme lisible : un indicateur ni consulté ni compris n'a aucune capacité de pilotage.

Le programme DevOps Research and Assessment (DORA) retient quatre indicateurs de performance de livraison : fréquence de déploiement, délai de mise en production, taux d'échec des changements et délai de rétablissement [3]. Ils supposent un produit déployé en continu, ce qui n'est pas le cas d'un équipement installé sur le terrain. Ce travail les adapte : fréquence d'intégration à la place de la fréquence de déploiement, délai de retour à la place du délai de mise en production, et délai de rétablissement d'une chaîne cassée plutôt que d'un service en production.

Ce que l'on retient. La supervision n'est pas nécessaire au fonctionnement de la chaîne, mais elle transforme des résultats ponctuels en information de pilotage, alimente le dossier de preuves attendu par le cadre normatif (section 3.2) et fournit la série de mesures sans laquelle aucune amélioration ne pourrait être démontrée au chapitre 5.

3.5 Synthèse et positionnement

Table 3.1 – Positionnement du travail par rapport à l'état de l'art

Pilier	Apport de la littérature	Reste à traiter
Intégration continue	Principes, bénéfices, obstacles de l'embarqué	Séparer applicatif et image ; fiabiliser l'environnement de compilation
Qualité et sécurité	Analyse statique, composition, cadre normatif	Restituer les résultats au développeur ; politique face à la dette
Infrastructure d'exécution	Exécuteurs éphémères, conteneurs	Composer avec l'absence de persistance
Observabilité	Indicateurs DORA	Adapter les indicateurs à un firmware ; historiser et visualiser

Chapitre 4

Analyse des besoins, conception et mise en œuvre des évolutions de la chaîne CI/CD

4.1 Analyse de l'existant et recueil des besoins

4.1.1 Situation à mon arrivée

À mon arrivée dans l'équipe COM, trois workflows GitHub Actions étaient en place : le premier compilait la couche applicative, exécutait les tests unitaires et l'analyse statique Coverity ; le deuxième construisait la distribution Linux embarquée avec Yocto ; le troisième produisait l'image de l'environnement de compilation. Cette organisation offrait un premier niveau d'automatisation, mais restait perfectible sur le suivi des indicateurs de qualité, la centralisation des résultats, l'observabilité des chaînes et l'automatisation des contrôles de sécurité. Ma mission consistait donc à reprendre cette chaîne, à en identifier les limites et à la faire évoluer, non à en concevoir une nouvelle.

4.1.2 Méthode de recueil

La prise en main a combiné trois sources. La documentation interne (Confluence) et l'étude des dépôts GitHub Enterprise (workflows, scripts, configuration, historique des modifications) ont permis de reconstituer les étapes de construction et les dépendances entre dépôts. Des échanges réguliers avec trois développeurs de l'équipe COM et le Product Owner ont confronté cette documentation au fonctionnement réel et fait émerger les attentes des utilisateurs. Enfin, mon tuteur, coordinateur DevSecOps, m'a aidé à distinguer ce qui était imposé par l'environnement de l'entreprise de ce qui pouvait être adapté au module de communication. Ce recueil s'est poursuivi de manière itérative tout au long du stage.

4.1.3 Besoins identifiés

L'analyse n'a révélé aucun dysfonctionnement bloquant : l'enjeu était d'accompagner la croissance des besoins du projet. Les besoins identifiés se regroupent en quatre axes :

- **Reproductibilité** : enrichir, optimiser et versionner l’image de compilation, et y intégrer le Software Development Kit (SDK) de la cible pour disposer d’un environnement commun à la compilation native et croisée ;
- **Automatisation** : prendre en charge plusieurs configurations (compilation native AMD64 pour les tests et la couverture, compilation croisée AArch64 pour le produit) et fiabiliser les scripts qui lancent les analyses, attendent leur fin, récupèrent les résultats et produisent les rapports ;
- **Qualité et sécurité** : mesurer la couverture des tests (lignes, fonctions, branches) et analyser la composition binaire de l’artefact livré avec BDBA pour identifier les composants tiers et leurs vulnérabilités ;
- **Observabilité** : dépasser la consultation d’une exécution isolée en collectant, historisant et restituant les indicateurs dans des tableaux de bord Grafana.

4.2 Formalisation des exigences

4.2.1 Parties prenantes

Les développeurs COM sont les utilisateurs directs de la chaîne ; le Product Owner en exploite les résultats pour le suivi du projet ; le coordinateur DevSecOps veille à la cohérence avec les pratiques de l’entreprise. Les services internes (exécuteurs, serveurs d’analyse, référentiel d’artefacts) n’expriment pas de besoin fonctionnel, mais imposent des règles d’utilisation (tableau 4.1).

Table 4.1 – Parties prenantes et attentes principales

Partie prenante	Attentes principales
Développeurs COM	Résultats rapides et lisibles sur la compilation, les tests, la couverture et les analyses de sécurité
Product Owner	Visibilité sur l’avancement, l’évolution de la qualité et les principaux risques
Coordinateur DevSecOps	Chaîne automatisée, maintenable, sécurisée et compatible avec les services de l’entreprise
Services internes	Respect des politiques de sécurité, des restrictions d’accès et des conditions d’utilisation

4.2.2 Exigences fonctionnelles

Les exigences fonctionnelles (tableau 4.2) sont regroupées par chaîne et pour la solution d’observabilité.

Table 4.2 – Exigences fonctionnelles

Réf.	La solution doit. . .
<i>Chaîne applicative</i>	
EF1	compiler l'application en natif AMD64 et en croisé AArch64
EF2	prendre en charge les modes DEBUG et RELEASE
EF3	exécuter l'analyse statique et récupérer les défauts détectés
EF4	soumettre l'artefact cible à l'analyse de composition binaire
EF5	exécuter les tests unitaires et produire un rapport
EF6	exécuter les tests d'intégration compatibles avec l'intégration continue
EF7	mesurer la couverture du code par les tests unitaires
EF8	publier une synthèse lisible des compilations, tests, couverture et analyses
EF9	produire un artefact rattaché à la révision, à la configuration et à l'environnement
<i>Chaîne de la distribution</i>	
EF10	construire automatiquement la distribution Linux embarquée
EF11	produire les variantes d'image, notamment de développement et de production
EF12	générer le SDK utilisé pour la compilation croisée
EF13	publier les images et le SDK dans le référentiel d'artefacts
<i>Chaîne de l'image de compilation</i>	
EF14	construire une image contenant compilateurs, bibliothèques, outils d'analyse et SDK
EF15	vérifier la présence et les versions des principaux outils
EF16	valider l'image candidate par une construction applicative complète
EF17	versionner et publier l'image validée dans un registre
<i>Observabilité</i>	
EF18	collecter automatiquement les indicateurs produits par les chaînes
EF19	transformer les indicateurs dans un format commun
EF20	historiser les données pour les comparer dans le temps
EF21	restituer les indicateurs dans des tableaux de bord Grafana
EF22	visualiser l'évolution des constructions, tests, couverture, défauts et vulnérabilités

4.2.3 Exigences non fonctionnelles

Les exigences non fonctionnelles (tableau 4.3) précisent les propriétés attendues de la solution.

Table 4.3 – Exigences non fonctionnelles

Réf.	Exigence
<i>Reproductibilité</i>	
ENF1	Versions des outils de compilation et d’analyse épinglées dans l’image
ENF2	Même image utilisée sur les postes et dans la chaîne d’intégration
ENF3	Artefacts rattachés à la révision, à la configuration et à la version de l’environnement
<i>Performance</i>	
ENF4	Durée des chaînes compatible avec le rythme de développement
ENF5	Configurations indépendantes exécutées en parallèle
ENF6	Caches partagés pour la construction de la distribution
ENF7	Optimisations sans perte de fiabilité ni de traçabilité
<i>Maintenabilité et réutilisabilité</i>	
ENF8	Logique placée dans des scripts indépendants de l’orchestrateur
ENF9	Workflows paramétrables et réutilisables
ENF10	Évolutions documentées pour l’équipe
ENF11	Composants génériques facilitant une mutualisation COM/MET
<i>Sécurité</i>	
ENF12	Aucun secret en clair dans les dépôts ni dans les couches de l’image
ENF13	Jetons injectés à l’exécution, de portée minimale
ENF14	Aucun secret exposé dans les journaux d’exécution
<i>Observabilité</i>	
ENF15	Données des différents outils transformées dans un format homogène
ENF16	Collecte automatique, sans intervention manuelle
ENF17	Indicateurs définis et documentés permettant de suivre la maturité du module
ENF18	Tableaux de bord lisibles et exploitables par l’équipe
<i>Infrastructure d’exécution</i>	
ENF19	Exécution sur des machines éphémères sans état local
ENF20	Dépendances aux chemins et caches locaux limitées ou externalisées
ENF21	Compatibilité avec les politiques réseau et de sécurité de l’entreprise

Deux sujets sont volontairement hors de ces exigences : l’apport de l’intelligence artificielle au diagnostic des échecs, traité comme un axe exploratoire, et la migration vers une chaîne commune aux équipes COM et MET, que la solution doit seulement faciliter (ENF11).

4.2.4 Contraintes et hypothèses

La plateforme de code source, l’orchestrateur, les ressources d’exécution et plusieurs outils d’analyse sont imposés par l’entreprise. Le travail consiste donc à adapter la chaîne à ces services, non à choisir une plateforme.

Infrastructure d’exécution. Les traitements s’exécutent sur des machines éphémères, détruites à la fin de chaque job. L’isolation est meilleure, mais aucun fichier, dépendance ou résultat intermédiaire n’est conservé : les constructions longues exigent un cache partagé, indépendant de la durée de vie des exécuteurs. Les fonctions de GitHub Actions qui supposent

une machine persistante ou un accès privilégié ne sont pas disponibles. Les chemins de travail varient, ce qui impose de les normaliser pour les outils qui corrént les défauts par chemin de fichier. Enfin, des règles encadrent les droits d'exécution, l'accès réseau et l'usage des secrets.

Distribution Linux embarquée. Sa construction mobilise un grand nombre de composants et dure longtemps sans réutilisation des résultats intermédiaires ; sa maîtrise dépend des caches de téléchargement et d'état partagé et d'un parallélisme adapté aux exécuteurs. Ses livrables (images et SDK) évoluent à un rythme différent du code applicatif, ce qui justifie une chaîne séparée.

Outils d'entreprise. Les outils sont imposés ; la marge de décision porte sur l'organisation et le paramétrage des workflows, la répartition des jobs, les scripts exécutables en local, la configuration des outils, la sécurisation des accès, la gestion des caches, la restitution des résultats et la collecte des indicateurs.

Hypothèses. Les dépôts, les services internes et les secrets sont disponibles depuis les exécuteurs autorisés ; les tests sur cible matérielle restent hors de la chaîne automatique ; les images et le SDK produits par la chaîne de distribution servent de référence pour la compilation croisée ; les indicateurs collectés sont représentatifs de la qualité du firmware.

4.3 Conception des évolutions de la chaîne CI/CD

4.3.1 Principes directeurs

Six principes ont guidé la conception des évolutions. Ils découlent directement des limites identifiées et des contraintes de l'infrastructure d'exécution.

P1 — La logique réside dans des scripts, pas dans le YAML

Les fichiers de workflow se limitent à l'orchestration (déclencheurs, matrices, dépendances entre jobs, injection des secrets). Tout traitement non trivial est placé dans le dépôt `bluebird/ci`, sous `scripts/` : `cov_build.sh` (phases capture, analyse et commit de Coverity), `extract_cov.sh` et `pyCovXtract.py` (extraction des défauts), `gcov_summary_md.sh` et `robot_summary_md.py` (synthèses Markdown), `BDBA_GetReport.py` et `BDBA_ExtractMetrics.py` (analyse de composition). Les scripts `build_pm77xx-app.sh` et `build_pm77xx-yocto.sh` servent de références pour rejouer une construction en local. Ce principe répond à ENF8 : un script se teste sur un poste, un fichier de workflow ne se teste qu'en poussant un commit.

P2 — Un seul environnement de compilation

La même image de conteneur, `bbbbuild_cov_ut`, sert à la compilation native amd64 et à la compilation croisée aarch64. Elle embarque la chaîne native (GCC 13.3.0, CMake 3.28.3), Coverity 2024.12.0 et le SDK de la cible installé dans `/usr/local/oe-core-x86_64`. Le basculement vers la cible se limite à sourcer le script `environment-setup-aarch64-evo-linux` du SDK avant l'appel à CMake (ENF1, ENF2).

P3 — Ne publier une image qu’après l’avoir éprouvée

Une image candidate est publiée sous l’étiquette `:test`, validée en lui faisant construire l’application complète par la chaîne `pm77xx-app`, puis seulement promue sous son étiquette définitive (EF16).

P4 — Des paramètres explicites, avec des valeurs par défaut sûres

Chaque entrée du workflow est reprise dans une variable d’environnement `INPUT_*` munie d’une valeur par défaut, par exemple `INPUT_APP_REPO_BRANCH: "${{ inputs.app_repo_branch || 'develop' }}"`. Les exécutions planifiées, pour lesquelles le contexte `inputs` est vide, se comportent ainsi comme une exécution manuelle aux valeurs par défaut (ENF9).

P5 — Les secrets n’existent que le temps de l’exécution

La clé de service Git, les jetons Coverity, BDBA, Artifactory et VictoriaMetrics sont fournis par le mécanisme `secrets` de la plateforme. Les fichiers d’authentification sont écrits avec les droits 0600, le fichier de connexion Coverity (`.cov_cnx.yml`) est supprimé dès la fin de l’extraction, et la construction de l’image utilise un montage SSH éphémère (ENF12 à ENF14).

P6 — Restituer le résultat là où le développeur regarde déjà

Chaque job écrit une synthèse dans `$GITHUB_STEP_SUMMARY` (tests, couverture, défauts, composants vulnérables). Le développeur lit son résultat dans la page du run, sans ouvrir Coverity, BDBA ou Grafana (EF8).

4.3.2 Contraintes imposées et décisions d’intégration

Le tableau 4.4 distingue, pour chaque brique, ce qui est imposé par l’environnement de l’entreprise et la décision d’intégration prise. La colonne de droite reprend les points qui relèvent de mon travail.

Table 4.4 – Éléments imposés et décisions d'intégration

Brique	Imposé	Décision d'intégration
Orchestration	GitHub Enterprise Actions; exécuteurs éphémères <code>orchestrator-runner</code> (service RaaS); conteneur lancé avec <code>-u root</code>	Trois workflows distincts; matrice DEBUG/RELEASE $\times$ amd64/aarch64; <code>workflow_call</code> pour réutiliser la chaîne applicative; jobs légers (rapports, BDBA, publication) déportés sur <code>ubuntu-22.04</code>
Analyse statique	Coverity 2024.12.0; serveur Coverity interne, avec un projet et un flux dédiés	Trois phases séparées; build instrumenté limité à DEBUG/aarch64; <code>-strip-path</code> pour stabiliser les chemins; vues P1 et P2 extraites par <code>pyCovXtract.py</code>
Composition	Black Duck Binary Analysis (BDBA)	Analyse de l'archive RELEASE/aarch64 uniquement; scan par <code>Foxy/action-bdba-scan</code> puis récupération du rapport et des métriques par deux scripts Python via l'API
Artefacts	Instance Artifactory de l'entreprise; un dépôt générique et un registre d'images	Arborescence <code>sdp-artifacts/</code> , <code>sdk/</code> , <code>bsp-images/</code> ; nommage <code><nom>-<version>-<horodatage></code> ; publication limitée à la branche <code>main</code> pour l'application
Distribution	Yocto, branche <code>scarthgap</code> du dépôt <code>evo-image-config</code>	Pilotage par kas 5.2 plutôt que par BitBake direct; une cellule de matrice par variante d'image; génération du SDK sur la variante de développement
Métriques	Instance VictoriaMetrics interne; authentification Basic	Action composite <code>upload-vm-metrics</code> ; deux formats d'import selon la source; étiquettes <code>bu</code> , <code>project</code> , <code>repo</code> , <code>branch</code> , <code>workflow</code>

Deux décisions méritent d'être explicitées.

Où placer l'instrumentation Coverity. Le build Coverity et le build de test sont deux constructions distinctes. Le build capturé par `cov-build` cible l'architecture livrée (aarch64) : les unités de compilation analysées correspondent au code qui part en production. Les tests et la couverture s'exécutent sur un build amd64 non instrumenté par Coverity, ce qui évite de superposer l'instrumentation `cov-build` et l'instrumentation `gcov`. Dans le workflow, les deux étapes de compilation sont mutuellement exclusives par une condition `if` portée par la matrice.

Intégrer le SDK dans l'image de compilation. Le SDK est produit par la chaîne Yocto, publié dans Artifactory, puis installé dans l'image de compilation par le `Dockerfile`.

Les deux chaînes sont ainsi couplées par un artefact versionné et non par un état partagé.

4.3.3 Architecture cible

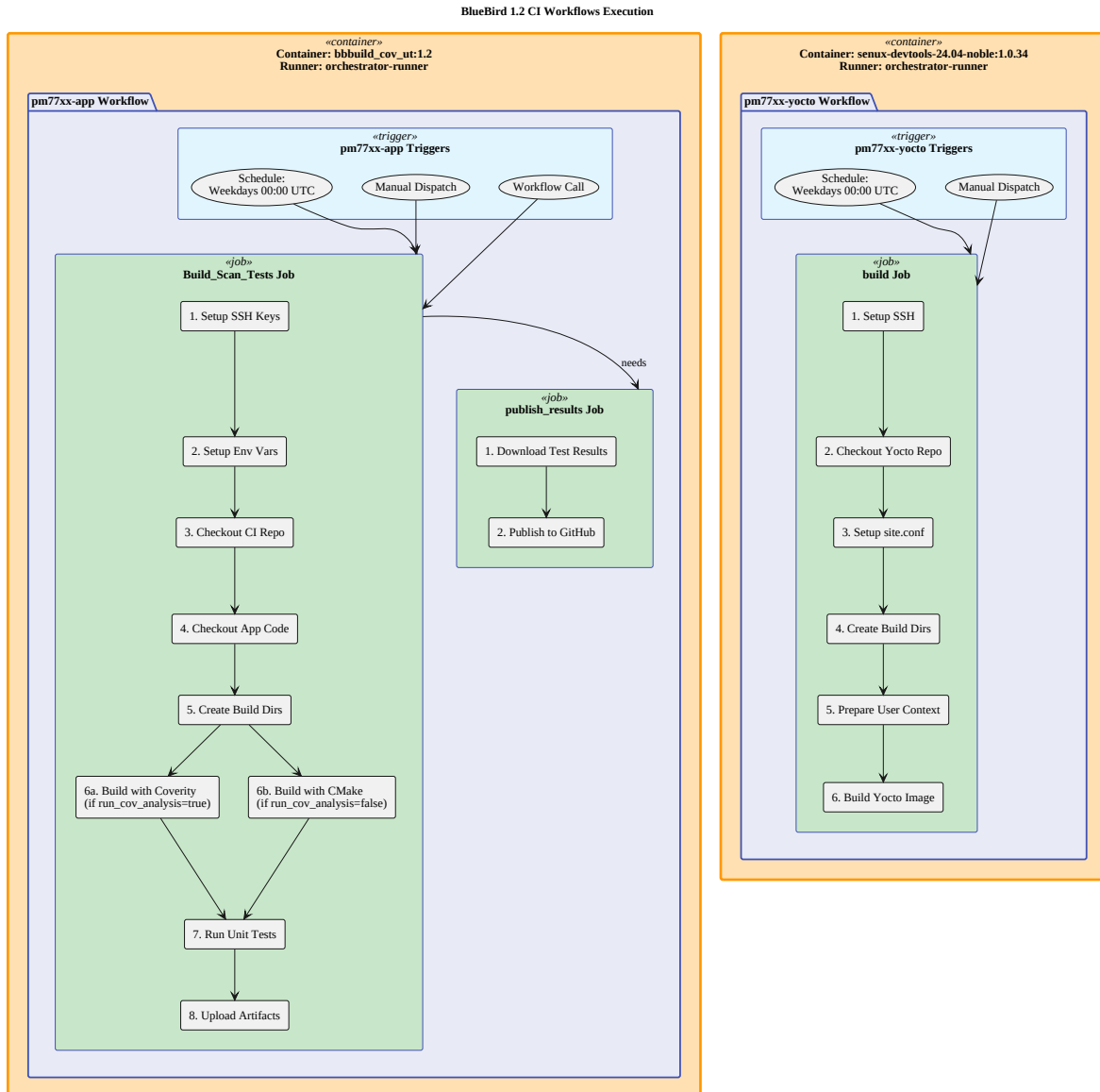


Figure 4.1 – Architecture générale de la chaîne d'intégration et de livraison

La figure 4.1 montre le flux. Un déclencheur (planifié en jours ouvrés, manuel ou appel depuis un autre workflow) démarre un exécuteur éphémère. Le job récupère l'image de compilation depuis Artifactory, puis, dans ce conteneur, extrait trois dépôts : `bluebird/ci` (scripts), `pm77xx-app/sdp` (code applicatif, sous-modules inclus) et `pm77xx-app/automation-tests` (suites Robot Framework). Les cellules de la matrice compilent, analysent et testent en parallèle ; les jobs aval téléchargent l'archive produite, génèrent les rapports, soumettent l'archive de production à BDBA, la publient dans Artifactory et envoient les métriques à VictoriaMetrics, lues par Grafana.

Le tableau 4.5 précise la part héritée et la part apportée.

Table 4.5 – Situation à mon arrivée et évolutions apportées

Domaine	Existant	Évolution apportée
Compilation	Compilation de l'application	Matrice <code>DEBUG/RELEASE</code> × native amd64 / croisée aarch64
Environnement	Image de compilation	Intégration du SDK ; optimisation du <code>Dockerfile</code> ; mise à jour des paquets
Qualité	Analyse statique Coverity ; tests unitaires	Couverture (<code>gcovr</code>) et synthèse ; tests d'intégration Robot Framework
Sécurité	—	Scan BDPA, rapport PDF et métriques
Livraison	Construction de la distribution	Publication des artefacts applicatifs, des images et du SDK
Observabilité	Résultats par exécution	Action de collecte vers VictoriaMetrics ; mise à jour du tableau de bord Grafana
Infrastructure	Exécuteurs éphémères	Étude des solutions NFS et snapshot ; correction de scripts bash et Python

4.3.4 Décomposition en chaînes

Le système comporte trois chaînes aux rythmes et aux durées différents (tableau 4.6). Le dépôt contient également un workflow `evo-bench-image`, destiné aux images du banc de test ; il est hors du périmètre de ce mémoire.

Table 4.6 – Chaînes d'intégration

Chaîne	Objet	Déclenchement
<code>pm77xx-app</code>	Compilation, analyses, tests et publication de la couche applicative	Cron <code>0 0 * * 1-5</code> , manuel, appelé
<code>pm77xx-yocto</code>	Cinq variantes d'image et SDK	Cron <code>0 22 * * 1-5</code> , manuel
<code>publish-image</code>	Construction, validation et publication de l'image de compilation	Manuel

La chaîne applicative est planifiée à minuit UTC et la chaîne Yocto à 22 h UTC : elles ne se disputent donc pas les exécuteurs. Les durées mesurées sont présentées au chapitre 5.

La séparation s'impose pour trois raisons. Premièrement, les rythmes diffèrent : le code applicatif change tous les jours alors que la distribution et l'image de compilation évoluent rarement. Deuxièmement, les durées sont incompatibles : intégrer une construction Yocto à plus d'une heure dans le chemin du développeur dégraderait le retour d'information (ENF4). Troisièmement, les chaînes s'articulent par des artefacts versionnés : le SDK sort de `pm77xx-yocto`, entre dans `publish-image`, et `publish-image` appelle `pm77xx-app` comme sous-workflow pour valider l'image candidate.

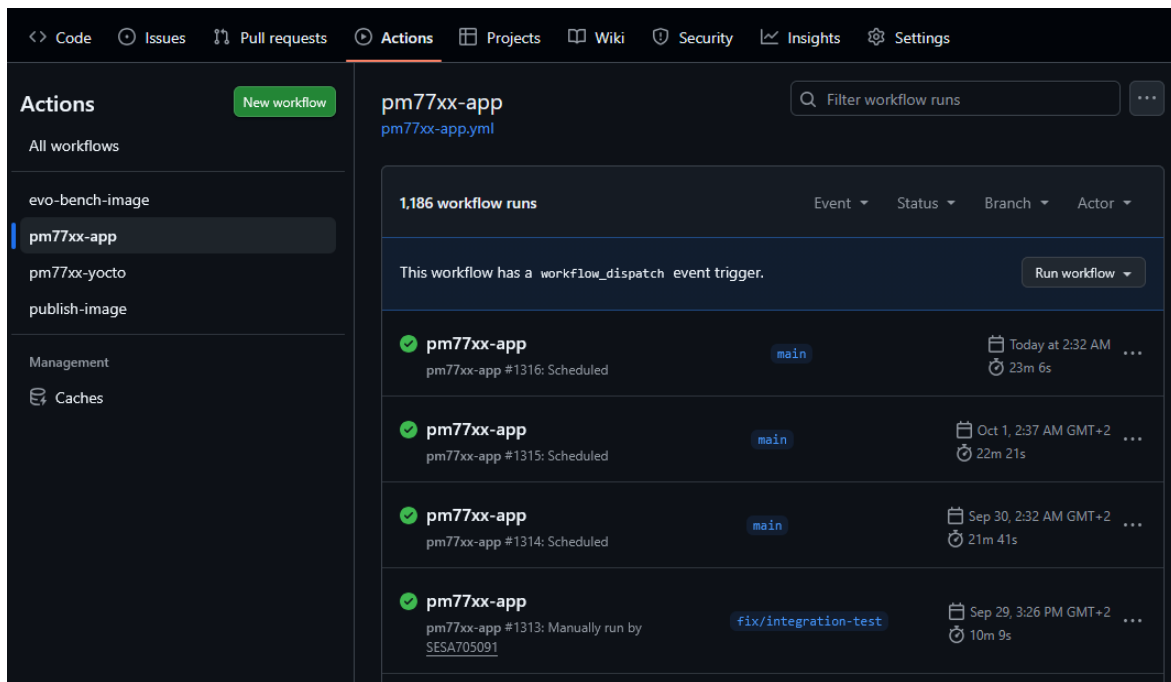


Figure 4.2 – Historique des exécutions de pm77xx-app depuis la création du workflow (1 186 exécutions)

4.4 Mise en œuvre technique

Chaque sous-section suit la même trame : l'objectif visé, l'état de départ, l'évolution apportée, puis la difficulté rencontrée. Les références EF et ENF renvoient aux exigences de la section précédente.

4.4.1 Les chaînes CI

Chaîne applicative pm77xx-app

Exigences : EF1 à EF9, ENF4, ENF5, ENF8, ENF9.

Paramètres. La chaîne accepte les entrées du tableau 4.7 par `workflow_dispatch` et `workflow_call`. Le mode appelé existe pour la validation de l'image de compilation : `publish-image` appelle `pm77xx-app` en lui passant l'étiquette candidate `bbbuid_cov_ut:test`, avec `run_cov_analysis=false` pour éviter de polluer le serveur Coverity avec une image non validée.

Table 4.7 – Paramètres d'entrée de `pm77xx-app`

Paramètre	Rôle	Défaut
<code>app_repo_branch</code>	Branche de <code>pm77xx-app/sdp</code>	<code>develop</code>
<code>test_repo_branch</code>	Branche de <code>automation-tests</code>	<code>main</code>
<code>builder_img_tag</code>	Image de compilation	<code>bbbuild_cov_ut:2.3</code>
<code>ci_repo_branch</code>	Branche de <code>bluebird/ci</code>	<code>main</code>
<code>run_cov_analysis</code>	Active la capture Coverity	<code>true</code>
<code>run_robot_tests</code>	Active les tests d'intégration	<code>false</code>
<code>workspace_path</code>	Répertoire de travail dans le conteneur	<code>/__w/ci/ci</code>

Matrice de construction. La construction est déclinée selon `build_type` ∈ {DEBUG, RELEASE} et `compiler_type` ∈ {amd64, aarch64}, soit quatre cellules indépendantes (`fail-fast: false`), exécutées en parallèle. Le tableau 4.8 répartit les activités sur ces cellules ; cette répartition est un arbitrage entre coût et couverture.

Table 4.8 – Répartition des activités sur la matrice

Activité	DEBUG	DEBUG	RELEASE	RELEASE
	amd64	aarch64	amd64	aarch64
Compilation	✓	✓	✓	✓
Analyse statique	—	✓	—	—
Tests unitaires	✓	—	✓	—
Couverture	✓	—	—	—
Tests d'intégration	✓*	—	—	—
Analyse de composition	—	—	—	✓
Publication	—	—	—	✓†

* si `run_robot_tests=true`. † sur la branche `main` uniquement.

Les tests s'exécutent sur amd64 parce que le binaire s'exécute nativement sur l'exécuteur ; les cellules aarch64 ne peuvent que compiler. La couverture exige l'instrumentation `gcov`, donc un build DEBUG. L'analyse de composition porte sur RELEASE/aarch64, c'est-à-dire sur l'artefact réellement livré. Le listing 4.1 montre le point de bascule entre compilation native et croisée.

```

1 - name: Build ${matrix.compiler_type} App in ${matrix.build_type}
  mode
2   id: build_app
3   if: ${env.INPUT_RUN_COV_ANALYSIS} == 'false'
4       || matrix.build_type != 'DEBUG'
5       || matrix.compiler_type != 'aarch64' }}
6   run: |
7     if [ "${matrix.compiler_type}" == 'aarch64' ]; then
8       . /usr/local/oecore-x86_64/environment-setup-aarch64-evo-linux
9     fi
10    cmake -DCMAKE_BUILD_TYPE=${matrix.build_type} \
11      -S "${env.SDP_ROOT}" \

```

```

12 -B "${ env.SDP_ROOT }}/${ matrix.build_type }-build.${{
13    matrix.compiler_type }}"
    cmake --build "." --parallel

```

Listing 4.1 – Compilation native ou croisée selon la cellule de matrice

La condition `if` rend cette étape exclusive de l'étape `cov_build_app`, qui porte la condition inverse. Le répertoire de build est suffixé par la configuration (`DEBUG-build.aarch64`), ce qui évite toute collision entre cellules.

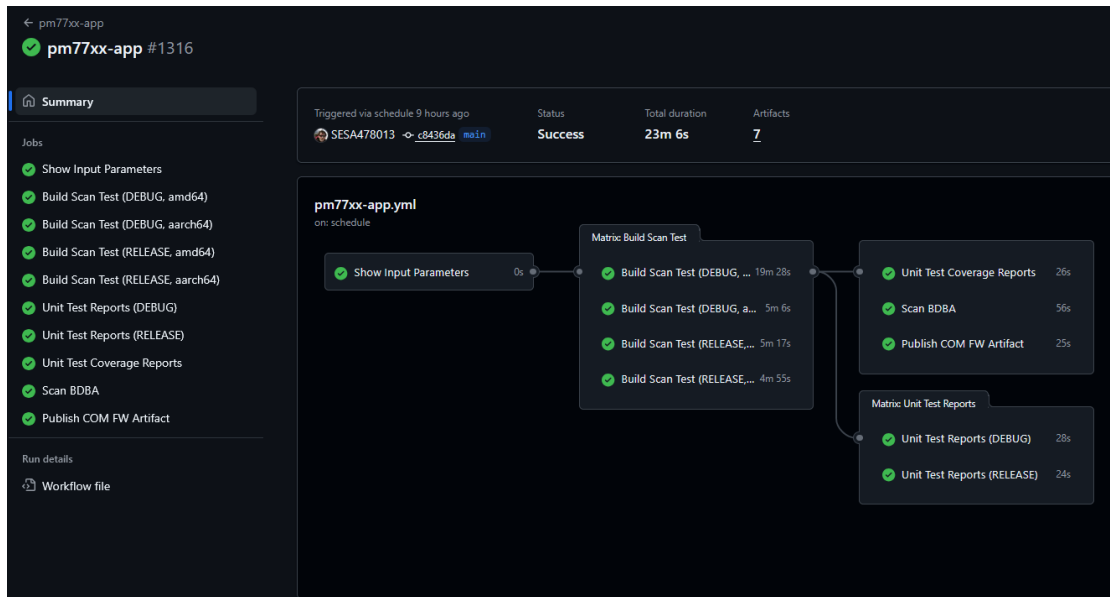


Figure 4.3 – Exécution de `pm77xx-app` (#1316) : matrice de construction et jobs aval

La figure 4.3 se lit de gauche à droite. Le job *Show Input Parameters* (0 s) écrit les paramètres dans la synthèse. Les quatre cellules de la matrice démarrent ensuite : la cellule `DEBUG/amd64` dure 19 min 28 s, contre 4 min 55 s à 5 min 17 s pour les trois autres ; elle cumule la compilation instrumentée, les tests, la couverture et, si activés, les tests d'intégration, et constitue donc le chemin critique des 23 min 06 s de l'exécution. Les jobs aval (couverture 26 s, BDBA 56 s, publication 25 s, rapports de test 28 s et 24 s) sont déclenchés dès la fin de la matrice (`needs: build-scan-test`).

Difficulté : clonage multi-dépôts. Le code est réparti entre un dépôt principal et de nombreux sous-modules privés. Le checkout utilise `submodules: recursive` avec la clé de service `GIT_SRV_SSH_KEY` et `persist-credentials: false`, de sorte que la clé ne reste pas dans la configuration Git du répertoire de travail.

Tests unitaires et couverture

Exigences : EF5, EF7, EF8.

Tests unitaires. Les tests sont lancés par `ctest`, avec export JUnit pour la publication, et excluent la suite `mcu_unit_tests`, destinée au cœur temps réel et inexécutable sur l'hôte :

```

1 SDP_ROOT="$SDP_ROOT" ctest \
2   --test-dir "$SDP_ROOT/${BUILD_TYPE}-build.${ARCH}" \
3   --exclude-regex "mcu_unit_tests" \
4   --output-on-failure \
5   --output-junit "$SDP_ROOT/${BUILD_TYPE}-build.${ARCH}/test_results.xml"

```

Listing 4.2 – Exécution des tests unitaires

L'étape est bornée par `timeout-minutes: 1` : un test qui ne rend jamais la main ne doit pas immobiliser l'exécuteur jusqu'à la limite globale. Le job `test-reports` publie ensuite les résultats avec l'action `EnricoMi/publish-unit-test-result-action` (13 tests réussis sur 13 dans l'exécution #1316).

Couverture. La couverture est mesurée avec `gcovr` sur la seule cellule `DEBUG/amd64` :

```

1 gcovr --root "$SDP_ROOT" --object-directory . \
2   --exclude '.*test/.*' \
3   --gcov-ignore-parse-errors=negative_hits.warn \
4   --html-details coverage/gcovr.html \
5   --lcov coverage/gcovr.info \
6   --json coverage/gcovr.json \
7   --json-summary coverage/gcovr-summary.json \
8   --print-summary | tee coverage/gcovr-summary.txt

```

Listing 4.3 – Mesure de couverture

Trois choix sont à relever. L'option `-gcov-ignore-parse-errors=negative_hits.warn` dégrade en avertissement les compteurs d'exécution négatifs produits par `gcov`, qui, sans cette option, interrompent la génération du rapport. Les quatre formats de sortie répondent à quatre usages : HTML pour la lecture, LCOV pour les outils tiers, JSON pour l'exploitation, texte pour la synthèse. Enfin, le pourcentage de lignes est extrait par un filtre `awk` sur la ligne `lines` : du résumé pour alimenter VictoriaMetrics.

`gcov_summary_md.sh` convertit le résumé en tableau Markdown avec badge et statut par métrique, selon deux seuils (50 % et 75 %). Sur le run #1316, la couverture est de 32,3 % des lignes (19 308 / 59 772), 20,9 % des fonctions (1 511 / 7 216) et 29,6 % des branches (5 559 / 18 775) ; les trois indicateurs sont sous le seuil bas. La chaîne rend donc visible un niveau de couverture faible plutôt que de le masquer.

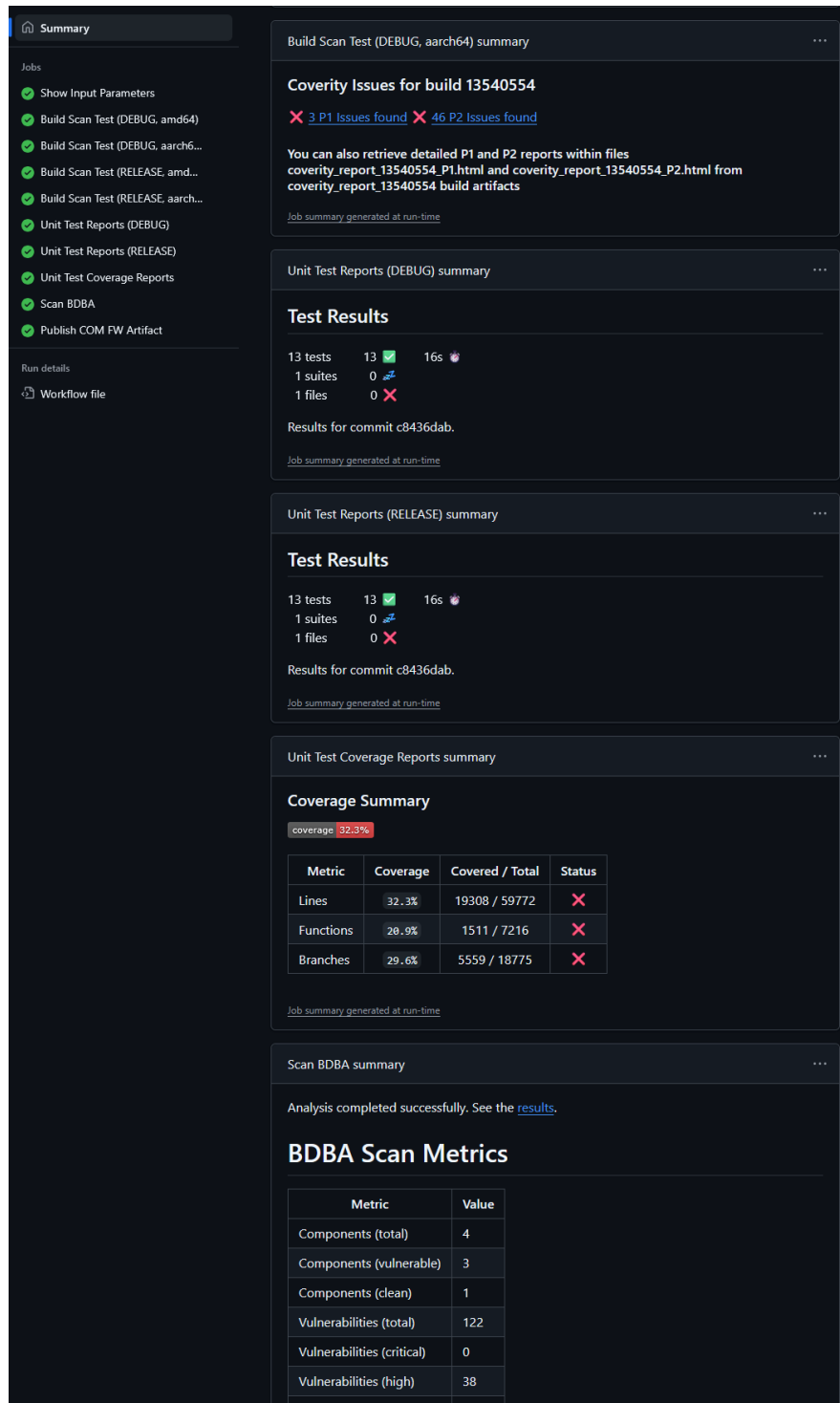


Figure 4.4 – Synthèse d'exécution : défauts Coverity, tests, couverture et BDBA

Difficulté : chemins d'artefacts. `upload-artifact` conserve l'arborescence absolue du conteneur. Le job `ut-coverage-reports`, qui s'exécute dans une autre image, doit donc retrouver les fichiers sous `./build-amd64/___w/ci/ci/pm77xx/pm77xx-app/sdp/DEBUG-build.amd64`. Ce chemin est centralisé dans une variable `BUILD_PATH` pour limiter sa duplication.

Tests d'intégration Robot Framework

Exigence : EF6.

Les suites Robot Framework du dépôt `automation-tests` exercent les services applicatifs. Leur exécution est un ajout récent. Elle est désactivée par défaut (`run_robot_tests=false`) et ne concerne que la cellule `DEBUG/amd64`, car le lanceur de services attend les binaires de l'arborescence `x86_64/DEBUG`. La séquence, entièrement conditionnée à `steps.launch_sdp.outcome`, est la suivante :

1. `cmake -install` copie configuration, scripts de migration de base de données et certificats sous `/opt/app-ext/sdp`;
2. `sdp.launch.in.container.sh` démarre les services en arrière-plan (`timeout-minutes: 5`);
3. `robot` produit `output.xml`, `log.html`, `report.html` et `xunit.xml`;
4. `sdp.launch.in.container.sh -kill` arrête les services dans une étape `if: always()`, même si les tests ont échoué;
5. `robot_summary_md.py` écrit la synthèse (badge de taux de réussite, compteurs, 50 échecs listés au plus) dans le résumé du run.

La difficulté principale est d'exécuter un ensemble de services système (`dbus`, services dépendant de `libsystemd`) dans un conteneur sans gestionnaire de services complet. Le lanceur démarre donc chaque service lui-même et la garde de 5 minutes protège contre un service qui resterait au premier plan.

Analyse statique Coverity

Exigence : EF3.

Coverity préexistait ; mon travail a porté sur la fiabilisation des scripts et sur la restitution. L'analyse suit trois phases pilotées par `cov_build.sh {capture|analyze|commit}`, chacune dans une étape distincte afin d'isoler les échecs et de mesurer chaque durée. La configuration `scripts/coverity_ci.yaml` active tous les vérificateurs, y compris la famille de sécurité C :

```

1 capture:
2   build:
3     cov-build-args: [--parallel-translate=4]
4 analyze:
5   c-cpp-virtual: true
6   c-cpp-fnptr: true
7   checkers: { all: true, rule: true, c-family-security: true }
8   cov-analyze-args:
9     - --strip-path
10    - /__w/ci/ci/pm77xx/pm77xx-app/
11 commit:
12   connect:
13     stream: "<flux-projet>"
14     url: "https://<serveur-coverity>"

```

Listing 4.4 – Configuration Coverity

`c-cpp-virtual` et `c-cpp-fnptr` résolvent les appels virtuels et les pointeurs de fonction, nombreux dans le code applicatif. `-strip-path` supprime le préfixe du répertoire de travail afin que l'identité d'un défaut (fichier, fonction, type) reste stable d'un run à l'autre malgré des exécuteurs éphémères (ENF20). Sans lui, un même défaut serait compté comme nouveau à chaque modification du chemin.

Restitution ciblée. `extract_cov.sh` interroge, pour chacune des vues P1 et P2, l'API REST `/api/v2/streams/viewContents/{viewid}` via `pyCovXtract.py`, génère les rapports `coverity_report_<run>_P1.html` et `_P2.html`, et expose `cov_p1_issue_nbr`, `cov_p2_issue_nbr` et `cov_total_issue_nbr` dans `GITHUB_OUTPUT`. Sur le run #1316 : 3 défauts P1 et 46 défauts P2.

Politique de blocage. Le script retourne 1 lorsque des défauts sont trouvés. L'étape l'invoque avec `|| true`, car un code de sortie non nul colorait l'étape en rouge alors que le résultat est déjà exposé dans la synthèse. Seuls un échec de capture, d'analyse ou de connexion fait donc échouer le job. Ce choix est assumé : avec une dette initiale de 46 défauts P2, un blocage immédiat paralyserait l'équipe. La trajectoire cible est un blocage sur les défauts nouvellement introduits uniquement, ce que l'option `-diff` de `pyCovXtract.py` permettrait de calculer.

Analyse de composition BDBA

Exigence : EF4.

Le job `scan-bdba` télécharge l'archive `RELEASE/aarch64` puis enchaîne trois étapes :

1. soumission par `Foxy/action-bdba-scan@v1.1.1` (`delete-binary: true`, produit `pm77xx-app` version 1.0.0) ;
2. `BDBA_GetReport.py` interroge `/api/product/{id}/` toutes les 30 s jusqu'au statut R (délai maximal de 600 s) puis télécharge le rapport PDF ;
3. `BDBA_ExtractMetrics.py` produit un fichier JSON Lines au format d'import VictoriaMetrics, et expose les compteurs `bdba_components_total`, `bdba_components_vuln`, `bdba_vulns_total`, `bdba_vulns_critical`, `bdba_vulns_high`, `bdba_vulns_medium`, `bdba_vulns_low`.

Le tableau 4.9 définit les indicateurs. Sur l'exécution capturée (figure 4.5), BDBA identifie 4 composants dont 3 vulnérables (`cjson`, `openldap`, `hostapd`) pour 122 vulnérabilités : 5 critiques, 19 élevées, 86 moyennes et 12 faibles. Les compteurs BDBA de la synthèse (figure 4.4) proviennent d'une autre exécution et diffèrent donc légèrement ; ce mémoire retient les valeurs du rapport.

Table 4.9 – Indicateurs extraits de l'analyse de composition

Indicateur	Définition
<code>bdba_component</code>	Composants tiers détectés, étiquetés <code>vulnerable</code> ou <code>clean</code>
<code>bdba_vulns</code>	Vulnérabilités connues, étiquetées par sévérité

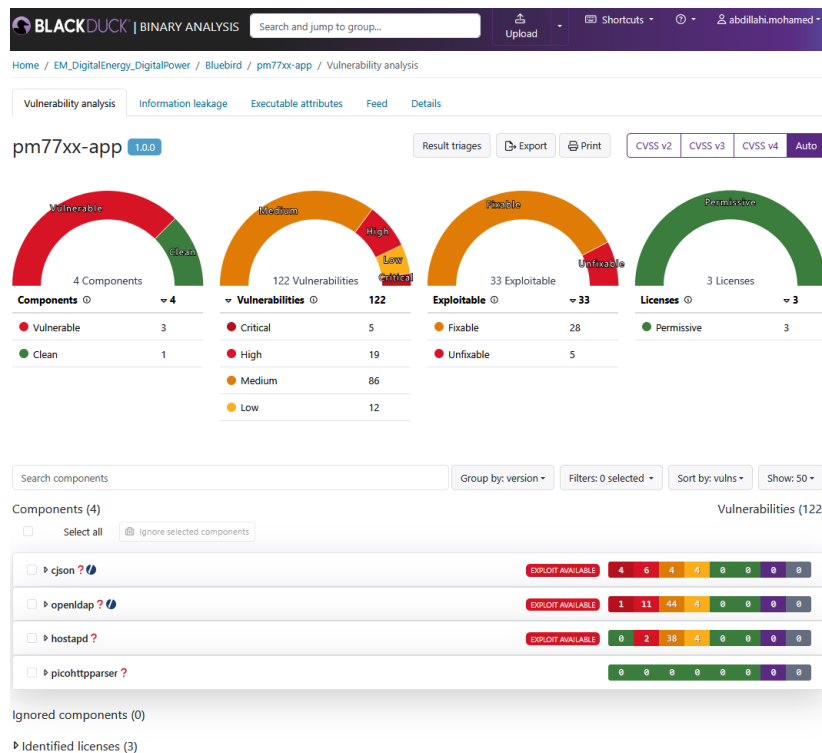


Figure 4.5 – Rapport BDBA de l’archive pm77xx-app (RELEASE/aarch64)

Cette analyse alimente l’inventaire des composants que les obligations de nomenclature logicielle exigent (section 3.2).

Publication des artefacts applicatifs

Exigence : EF9.

Chaque cellule empaquette son résultat avant de le publier : `DESTDIR=/tmp/staging` `cmake -install` installe dans une arborescence temporaire, puis `tar -czf` archive lib et bin depuis `/opt/app-ext/sdp`. L’archive est conservée comme artefact de run (`<BUILD>-build-<ARCH>`). Pour la branche `main`, le job `upload-artifact` publie l’archive `RELEASE/aarch64` dans Artifactory par `jf rt upload` dans le répertoire `sdp-artifacts/` du dépôt générique, sous un nom de la forme `pm77xx-app-artifact-<version>-<horodatage>.tar.gz`. La taille de l’archive est aussi mesurée (du `-cb`) et envoyée comme métrique `sdp_artifact_size` (78,0 Mo sur la capture du tableau de bord).

Limite : la version `ARTIFACT_VERSION` est fixée à `0.0.1`; seul l’horodatage distingue deux artefacts, et le nom ne contient ni l’empreinte du commit ni la version de l’image de compilation. L’exigence EF9 n’est donc que partiellement couverte.

Chaîne de la distribution pm77xx-yocto

Exigences : EF10 à EF13, ENF6.

La configuration est décrite avec kas 5.2 en trois niveaux, composés dans la ligne de commande par des « : » :

```
1 kas build kas/board/<carte>.yaml:kas/project/image-evo-dev.yaml
2 kas build kas/board/<carte>.yaml:kas/project/image-evo-dev.yaml:\
3   kas/project/option/populate-sdk.yaml
```

Listing 4.5 – Appels kas : image et SDK

Le premier niveau décrit la carte (la carte cible), le second le projet d'image, le troisième des options, dont `populate-sdk.yaml`. Pour la variante de production, le fichier de carte est suffixé (`<carte>-prod.yaml`) grâce à l'entrée `include` de la matrice. Les cinq variantes (tableau 4.10) sont construites en parallèle.

Table 4.10 – Variantes d'image produites

Variante	Usage
<code>image-evo-dev</code>	Développement, outils de débogage ; sert aussi à générer le SDK
<code>image-evo-hwtest</code>	Validation matérielle en production
<code>image-evo-netboot</code>	Démarrage réseau (initramfs)
<code>image-evo-rescue</code>	Image de secours (initramfs)
<code>image-evo-prod</code>	Production

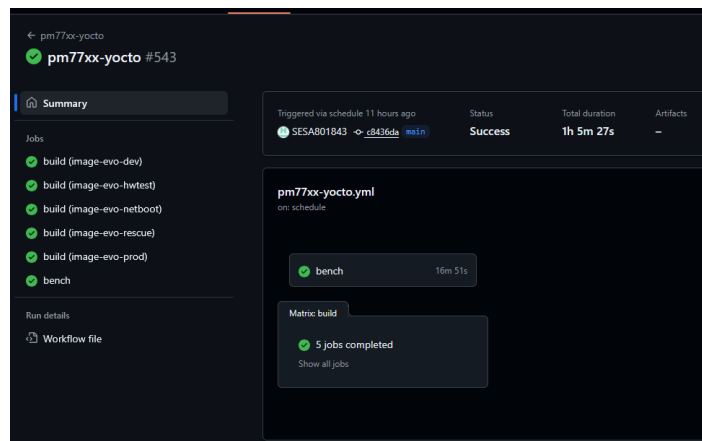


Figure 4.6 – Exécution de pm77xx-yocto (#543, 1 h 05 min 27 s)

Difficulté : BitBake refuse de s'exécuter en root. Le conteneur est imposé en root par l'infrastructure (section 4.4.2), or BitBake refuse de s'exécuter sous cet utilisateur. Le workflow écrit donc la clé SSH sous `/home/user/.ssh`, transfère la propriété du répertoire de travail à `user` (`chown -R`), déclare le dépôt en `safe.directory` puis lance `kas` par `su user -c`.

Publication. Après la construction, le SDK est localisé par `find build/*tmp*/deploy/sdk -name '*-toolchain-*.sh'` puis publié dans `sdk/` ; les variantes `dev` et `prod` sont archivées

(`find -type f | tar -transform 's|.*|/|'` met tous les fichiers à la racine de l'archive) et publiées dans `bsp-images/`. Pour la variante dev, une archive « flashable » regroupe `deploy/images` et le fichier `machine <carte>.conf` dans `bsp-images/standalone-flashing-images/`. Un développeur applicatif obtient ainsi un compilateur croisé et une image de référence sans reconstruire la distribution.

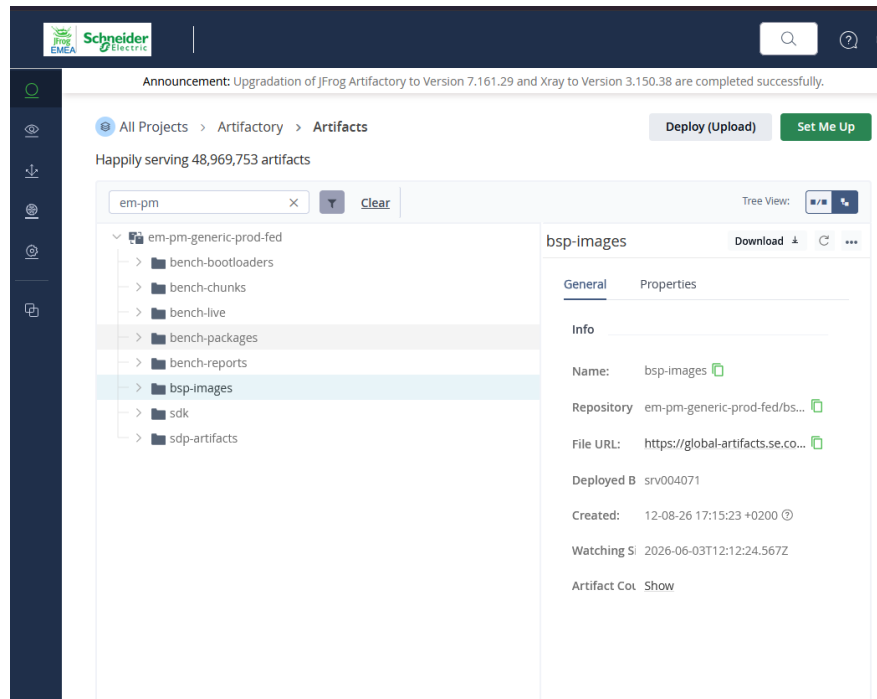


Figure 4.7 – Organisation du dépôt générique Artifactory

Chaîne de l'image de compilation `publish-image`

Exigences : EF14 à EF17, ENF1, ENF2, ENF12.

Construction du Dockerfile. L'image est construite en trois étages à partir d'une image Ubuntu 24.04 (`se-ubuntu-noble:3.0.1`) : `compiler` (paquets et compilateurs), `cov-sdk-builder` (Coverity et SDK) et `app_builder` (image finale). Seul le contenu utile est copié dans l'étage final (`COPY -from=cov-sdk-builder`), de sorte que l'archive Coverity et l'installateur du SDK n'alourdissent pas l'image.

Table 4.11 – Principaux composants de l’image et versions épinglées

Composant	Rôle	Version
GCC / G++	Compilation C et C++	13.3.0-6ubuntu2~24.04.1
CMake	Système de construction	3.28.3-1build7
Meson / Ninja	Brique de cybersécurité (csb-app)	1.3.2 / 1.11.1
Coverity Analysis	Analyse statique	2024.12.0
GoogleTest / GMock	Tests unitaires	1.14.0-1
CMocka	Doublures de dépendances	1.1.7-3
gcovr	Couverture	7.0-1
Doxygen / Graphviz	Documentation	1.9.8 / 2.42.2
Python	Scripts d’intégration continue	3.12.3

Les optimisations apportées sont les suivantes :

- versions épinglées par paquet (`cmake=3.28.3-1build7`), pour que deux constructions séparées dans le temps utilisent la même chaîne d’outils (ENF1) ;
- `-no-install-recommends`, qui évite l’installation de paquets non demandés ;
- caches `BuildKit --mount=type=cache,target=/var/cache/apt` et `/var/lib/apt` (`sharing=locked`) : les paquets déjà téléchargés sont réutilisés d’une construction à l’autre sans rester dans une couche ;
- construction multi-étages, décrite plus haut.

Accès aux dépôts privés. La brique de cybersécurité `csb-app` (version `Release_csbApp_2.6.0`) est clonée et compilée avec Meson dans l’image. Le clonage utilise `RUN --mount=type=ssh`, alimenté par `docker build --ssh default` : la clé reste dans l’agent SSH de l’hôte et n’est écrite dans aucune couche de l’image.

```

1 COPY ./resources/*sdk-installer*.sh ${SDK_PATH}/sdk-installer.sh
2 RUN chmod +x sdk-installer.sh \
3     && ./sdk-installer.sh -d /usr/local/oecore-x86_64 -y
4
5 RUN --mount=type=ssh \
6     ssh-keyscan <forge-interne> >> /root/.ssh/known_hosts \
7     && git clone --branch Release_csbApp_2.6.0 \
8     git@<forge-interne>:<organisation>/csb-app.git csb-app

```

Listing 4.6 – Installation du SDK et clonage sécurisé

L’image définit l’utilisateur `user`, mais son exécution en intégration se fait en `root` (`options: -u root`) pour des raisons de droits d’écriture imposées par RaaS ; ce point est reporté en perspectives.

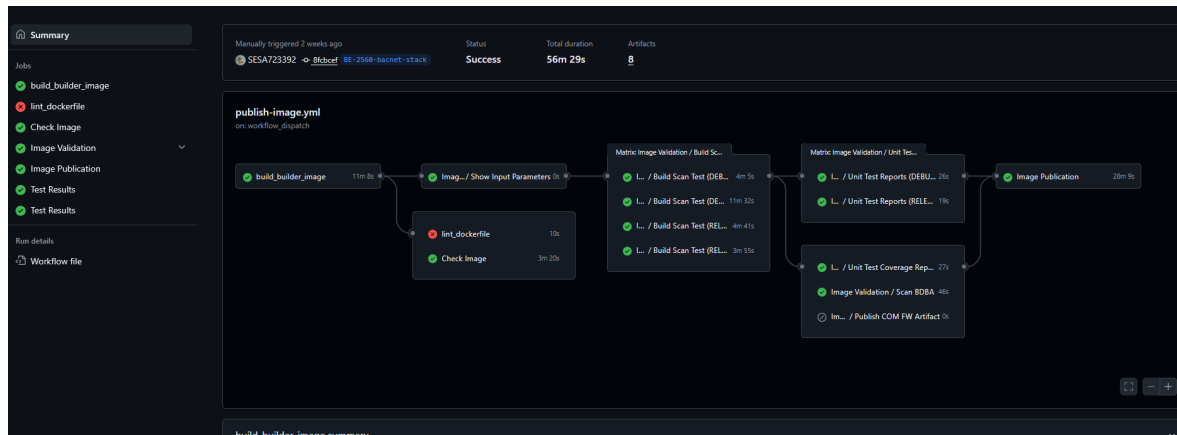


Figure 4.8 – Exécution de publish-image : construction, contrôles, validation, publication

Cycle de validation. La figure 4.8 détaille l’enchaînement :

1. `build_builder_image` (11 min 08 s) construit avec `DOCKER_BUILDKIT=1 docker build -ssh default -build-arg COV_VERSION=2024.12.0` et pousse l’image `:test` ;
2. `lint_dockerfile` exécute `hadolint`, et `Check Image` (3 min 20 s) lance `display_version.sh` dans l’image pour vérifier la présence et les versions des outils (EF15) ;
3. *Image Validation* appelle `pm77xx-app` avec `builder_img_tag=bbbbuild_cov_ut:test` : quatre cellules de compilation, rapports de test, couverture et BDBA ;
4. *Image Publication* (28 min 09 s) revérifie l’étiquette, republie l’image sous son étiquette définitive et en exporte une archive.

Une image n’atteint donc l’étiquette définitive qu’après avoir prouvé qu’elle construit le produit. Sur la capture, le job `lint_dockerfile` est en échec alors que l’exécution se termine en succès : les erreurs de `hadolint` sont tolérées. C’est un choix provisoire, le temps de résorber les avertissements existants.

4.4.2 L’infrastructure d’exécution

Exigences : ENF4, ENF6, ENF19 à ENF21.

Modèle d’exécuteurs éphémères

Les jobs lourds (compilation, Yocto, construction d’image) s’exécutent sur `orchestrator-runner`, un service d’exécuteurs qui alloue une machine neuve par job et la détruit ensuite. Les jobs légers (rapports de test, couverture, BDBA, publication) utilisent `ubuntu-22.04` avec l’image `se-ubuntu-noble:3.0.1` : ils n’ont pas besoin de la chaîne d’outils et ne doivent pas occuper un exécuteur coûteux. Les contraintes générales de ce modèle sont décrites en section 4.2.4 ; leur traduction dans la chaîne est la suivante :

- **Droits.** Le conteneur est lancé avec `options: -u root`, ce qui oblige la chaîne Yocto à rétrograder les privilèges (section 4.4.1).
- **Chemins.** Le répertoire de travail `/_w/ci/ci` est paramétré (`workspace_path`) et neutralisé côté Coverity par `-strip-path`.
- **Temps.** Des garde-fous `timeout-minutes` limitent les étapes susceptibles de bloquer.

Limite : le cache de la distribution

Les répertoires `DL_DIR` et `SSTATE_DIR` sont créés dans l'espace de travail du job par un fichier `site.conf` généré à l'exécution :

```
1 DL_DIR      = '$GITHUB_WORKSPACE/projets/yocto/downloads'
2 SSTATE_DIR = '$GITHUB_WORKSPACE/projets/yocto/sstate-cache'
3 BB_NUMBER_THREADS = '32'
4 PARALLEL_MAKE = '-j 32'
```

Listing 4.7 – Configuration Yocto générée par le job

L'exécuteur étant détruit à la fin du job, ces deux caches repartent de zéro à chaque exécution : chacune des cinq variantes retélécharge les sources et recompile l'ensemble des paquets, ce qui explique en grande partie la durée de 1 h 05 de la chaîne. La parallélisation à 32 threads ne réduit que la durée de chaque compilation, pas la redondance entre cellules. L'exigence ENF6 (cache partagé) n'est donc *pas* couverte par la chaîne actuelle.

Étude des solutions NFS et snapshot

J'ai étudié deux solutions pour rendre ces caches persistants malgré le caractère éphémère des exécuteurs (tableau 4.12).

Table 4.12 – Solutions étudiées pour le cache de la distribution

Solution	Principe	Points d'attention
Partage NFS	Montage d'un volume réseau comme <code>DL_DIR</code> et <code>SSTATE_DIR</code>	Écritures concurrentes des cinq cellules de la matrice ; latence sur de très nombreux petits fichiers ; disponibilité du service depuis les exécuteurs
Snapshot	Volume restauré au démarrage de l'exécuteur à partir d'un instantané du cache	Instantané à rafraîchir régulièrement ; cache en lecture seule, les nouveaux états n'étant pas réinjectés ; compatibilité avec le service d'exécuteurs

Conclusion de l'étude. La solution retenue est le partage NFS. Le snapshot ne fournit qu'un cache en lecture seule, figé à la date de l'instantané : les états produits par une exécution ne profitent pas aux suivantes tant que l'instantané n'est pas régénéré. Un volume NFS, au contraire, est alimenté à chaque construction et reste à jour sans opération supplémentaire. Le principal point à traiter lors de la mise en œuvre est la gestion des écritures concurrentes des cinq cellules de la matrice. Cette solution n'est pas encore mise en œuvre ; elle constitue la première perspective de ce travail (section 6.4).

Gestion des secrets et sécurité

Les secrets utilisés sont `GIT_SRV_SSH_KEY`, `COV_AUTH_JSON`, `COV_SRV_CNXON`, `AM_BDBA_API_TOKEN`, `JFROG_SRV_PAT` et `VMET_PASSWORD`. Les fichiers qui les contiennent sont créés avec `chmod 0600` et supprimés après usage (`.cov_cnx.yml`), le checkout n'enregistre pas les

identifiants (`persist-credentials: false`) et l'image de compilation ne contient aucune clé grâce au montage SSH. Deux limites subsistent : le jeton BDBA est passé en argument de ligne de commande aux scripts Python, donc visible dans la liste des processus de l'exécuteur (masqué dans les journaux, mais pas dans la machine) ; et la chaîne Yocto écrit la clé SSH sur le disque de l'exécuteur, dans un répertoire détruit avec lui.

4.4.3 L'observabilité

Exigences : EF18 à EF22, ENF15 à ENF18.

La synthèse d'un run ne montre pas de tendance. Pour suivre la qualité dans le temps, chaque chaîne envoie ses indicateurs à VictoriaMetrics, qu'un tableau de bord Grafana interroge.

Action de collecte. L'action composite `.github/actions/upload-vm-metrics` encapsule l'envoi pour éviter de dupliquer les appels `curl` dans chaque workflow. Elle accepte deux modes, selon la source :

```

1 # Mode fichier : JSON Lines d'import (BDBA, plusieurs séries)
2 curl -T "$METRICS_JSONL_FILE" -u "$VMET_USER:$VMET_PASSWORD" \
3     -X POST "https://$VMET_URL/api/v1/import"
4
5 # Mode valeur unique : format texte Prometheus
6 curl -d "${METRIC_NAME}${METRIC_LABELS} ${METRIC_VALUE}" \
7     -u "$VMET_USER:$VMET_PASSWORD" \
8     -X POST "https://$VMET_URL/api/v1/import/prometheus"

```

Listing 4.8 – Deux modes d'envoi des métriques

Le mode texte suffit pour une valeur scalaire (taille de l'artefact, couverture) ; le mode JSON Lines est nécessaire quand un script produit plusieurs séries étiquetées (`bdba_component` par statut, `bdba_vulns` par sévérité). Les étiquettes communes (`bu="em"`, `project="bluebird"`, `repo`, `branch`, `workflow`) permettent de filtrer par chaîne et par branche dans Grafana. Le nom d'utilisateur est une variable de dépôt et le mot de passe un secret.

Table 4.13 – Indicateurs collectés vers VictoriaMetrics

Métrique	Contenu	Origine
<code>sdp_artifact_size</code>	Taille de l'archive <code>lib + bin</code>	<code>du -cb</code> , chaque cellule
<code>code_coverage</code>	Couverture de lignes (%)	<code>gcovr</code> , <code>DEBUG/amd64</code>
<code>bdba_component</code>	Composants, par statut	<code>BDBA_ExtractMetrics.py</code>
<code>bdba_vulns</code>	Vulnérabilités, par sévérité	<code>BDBA_ExtractMetrics.py</code>

Tableau de bord. J'ai mis à jour le tableau de bord Grafana « Bluebird w1.2 / Daily – COM team » (figure 4.9). Il est organisé en quatre sections : sprint en cours, *Quality & Security* (défauts Coverity P1 et P2, couverture, taille de l'artefact, répartition des composants et des vulnérabilités), état des workflows (statut et durée du dernier run, un onglet par chaîne) et

pull requests ouvertes. À la date de la capture, il affiche 3 défauts P1, 46 défauts P2, 32,3 % de couverture, un artefact de 78,0 Mo et un dernier run `pm77xx-app` réussi en 23,1 minutes.

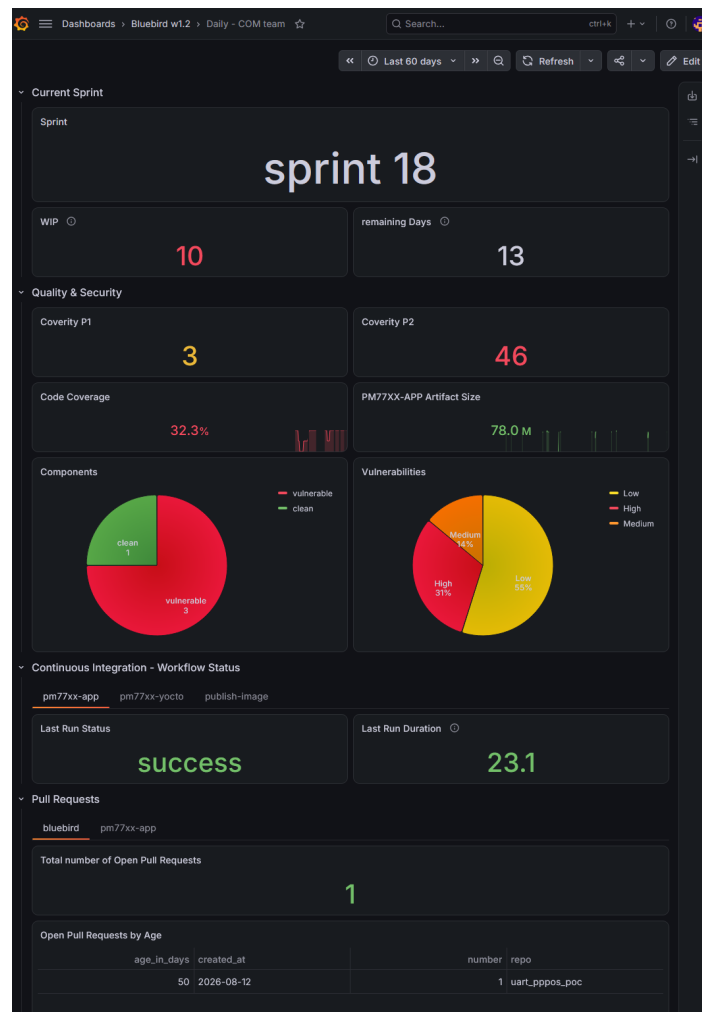


Figure 4.9 – Tableau de bord Grafana de l'équipe COM

4.4.4 Bilan

Couverture des exigences

Le tableau 4.14 indique, pour chaque groupe d'exigences fonctionnelles, la section qui les traite et leur état.

Table 4.14 – Couverture des exigences fonctionnelles

Réf.	Constat	Section	État
EF1, EF2	Matrice native/croisée × DEBUG/RELEASE	4.4.1	Couverte
EF3	Coverity en trois phases, rapports P1/P2	4.4.1	Couverte
EF4	Scan BDBA de l'archive RELEASE/aarch64	4.4.1	Couverte
EF5, EF7, EF8	Tests unitaires, couverture, synthèse	4.4.1	Couverte
EF6	Robot Framework, désactivé par défaut, DEBUG/amd64	4.4.1	Partielle
EF9	Version fixe, pas d'empreinte de commit dans le nom	4.4.1	Partielle
EF10 à EF13	Cinq variantes, SDK, publication	4.4.1	Couverte
EF14 à EF17	Image versionnée, validée, publiée	4.4.1	Couverte
EF18, EF20, EF21	Collecte, historisation, Grafana	4.4.3	Couverte
EF19, EF22	Deux formats d'import ; évolution des constructions limitée au dernier run	4.4.3	Partielle

Pour les exigences non fonctionnelles, ENF1, ENF2, ENF4, ENF5, ENF8, ENF9, ENF12 à ENF14 et ENF19 à ENF21 sont couvertes par les choix décrits plus haut. ENF3 (traçabilité des artefacts) est partielle, pour la raison donnée pour EF9. ENF6 (cache partagé de la distribution) n'est pas couverte : l'étude NFS/snapshot a conduit à retenir la solution NFS. ENF11 (mutualisation entre les équipes COM et MET) n'a pas été traitée.

Difficultés transverses

Le débogage des scripts bash et Python a occupé une part notable du travail. Les problèmes récurrents étaient :

- un code de sortie significatif (`extract_cov.sh`) interprété comme un échec par la plateforme ;
- des chemins qui varient selon l'image dans laquelle s'exécute le job (artefacts, répertoire de travail) ;
- des étapes qui ne terminent jamais, d'où les `timeout-minutes` ;
- des citations de secrets et de chaînes dans les scripts embarqués dans le YAML, déplacés dans `scripts/` pour être testables (principe P1).

Les limites qui découlent de ce bilan sont discutées en section 5.5, et les perspectives correspondantes en section 6.4.

Chapitre 5

Évaluation et résultats

5.1 Protocole d'évaluation

5.1.1 Questions d'évaluation

L'évaluation décline la question de recherche posée en section 1.2 en quatre questions, qui correspondent aux axes d'évolution du chapitre 4 (reproductibilité, automatisation, qualité et sécurité, observabilité) :

- QE1** L'environnement de construction obtenu est-il effectivement reproductible ?
- QE2** L'intégration des analyses de sécurité améliore-t-elle la détection des défauts sans dégrader la vitesse ?
- QE3** Les indicateurs collectés permettent-ils de piloter la qualité ?
- QE4** L'infrastructure d'exécution est-elle maîtrisée en coût et en effort d'administration ?

5.1.2 Sources de mesure

Table 5.1 – Sources des mesures utilisées

Indicateur	Source	Période
Durée et issue des exécutions	API de GitHub Actions, via <code>scripts/collecte-mesures.sh</code>	11 août – 17 sept. 2026
Couverture de code	Résumé <code>gcovr</code> et tableau de bord Grafana	Exécution #1316
Défauts d'analyse statique	Vues Coverity P1 et P2	Date de la capture
Vulnérabilités de composants	Rapport BDBA de l'archive RE-LEASE/aarch64	Date de la capture
Coût d'exécution	Estimation à partir des durées de jobs	—

Les durées et les issues proviennent de 164 exécutions des trois chaînes décrites au chapitre 4 (108 pour `pm77xx-app`, 51 pour `pm77xx-yocto`, 5 pour `publish-image`) sur cinq semaines. Sont retenues les exécutions planifiées, manuelles et appelées, sur toutes les branches : la chaîne a été développée en même temps qu'elle était mesurée, et les exécutions de branches de travail font partie de l'activité observée.

Les exécutions annulées ne sont comptées ni comme succès ni comme échec (3 pour `pm77xx-app`, 1 pour `pm77xx-yocto`). Les statistiques de durée utilisent la médiane plutôt que la moyenne, car deux exécutions planifiées de `pm77xx-app` durent 137,3 min (21 août) et 96,6 min (7 septembre), contre 20 à 26 min en général. Ces valeurs ne sont pas expliquées par le code et sont vraisemblablement liées à la disponibilité des exécuteurs ; elles sont conservées dans le relevé de l'annexe 6.5.

5.1.3 Validité

Trois limites encadrent l'interprétation.

Validité interne. Les chaînes ont évolué pendant la période d'observation (refactorisation de `pm77xx-app`, ajout de la collecte de métriques, publication des images, intégration de Robot Framework). Une variation de durée ou de taux d'échec ne peut donc pas être attribuée à une cause unique. Aucune mesure antérieure à mon arrivée n'existe : les indicateurs n'étaient pas historisés avant la mise en place de la collecte.

Validité externe. Il s'agit d'un produit et d'une équipe. Les durées dépendent du service d'exécuteurs de l'entreprise et du volume de code de ce firmware ; elles ne se transposent pas telles quelles.

Fiabilité. Les mesures issues de l'API sont rejouables avec le script de collecte. Les valeurs relevées dans les interfaces (couverture, défauts, vulnérabilités) sont des instantanés datés, non rejouables.

5.2 Impact

Cette section restitue les effets de la chaîne au-delà de la seule mesure technique : indicateurs de performance de livraison, coût d'exécution et apports pour l'équipe. Faute de mesure antérieure à la mise en place de la collecte, les valeurs « avant » ne sont pas disponibles ; seul l'état observé pendant la période est donc présenté.

5.2.1 Indicateurs de performance de livraison

Table 5.2 – Indicateurs de performance adaptés au contexte embarqué

Indicateur	Adaptation retenue	Avant	Observé (11 août – 17 sept.)
Fréquence d'intégration	Nombre d'exécutions de la chaîne applicative par semaine	n.d.	108 exécutions en cinq semaines, soit environ 20 par semaine
Délai de retour	Durée de la chaîne applicative complète	n.d.	Médiane de 21,8 min
Taux d'échec	Part des exécutions en échec, toutes branches	n.d.	17,6 % (19 sur 108) pour <code>pm77xx-app</code> , 17,6 % (9 sur 51) pour <code>pm77xx-yocto</code>
Délai de rétablissement	Délai jusqu'à la prochaine exécution planifiée réussie	n.d.	Au plus la nuit suivante pour les cinq échecs planifiés observés

Ces indicateurs sont adaptés de ceux de la section 3.4. Le débit d'intégration est approché par le nombre d'exécutions de la chaîne applicative, faute de notion de déploiement en production pour un firmware. Le délai de retour remplace le délai de mise en production par la durée entre le déclenchement et le résultat complet. Le taux d'échec est brut : il inclut les exécutions de branches où la chaîne elle-même était en cours de mise au point (section 5.3) et ne mesure donc pas uniquement la qualité du code. Pour le rétablissement, les cinq échecs des exécutions planifiées sur `main` (applicative les 11 août, 17 août et 1^{er} septembre, distribution les 2 et 8 septembre) ont chacun été suivis d'une exécution réussie la nuit suivante ; le temps de correction proprement dit n'est pas mesuré.

5.2.2 Coût et effort d'administration

Coût d'exécution. Les exécutions planifiées de la chaîne applicative cumulent environ 35 min de temps d'exécuteur par nuit (somme des quatre cellules de la matrice : 19 min 28 s, 5 min 06 s, 5 min 17 s et 4 min 55 s). Celles de la distribution en cumulent environ 5 h 25 (cinq cellules d'environ 65 min). Sur les cinq nuits ouvrées, cela représente de l'ordre de 30 heures de temps d'exécuteur par semaine, hors jobs légers et hors exécutions manuelles. Il s'agit d'une estimation à partir de durées de jobs, non d'une facturation. La distribution représente plus de 90 % de ce total, ce qui désigne le cache de construction (section 5.4.2) comme principal levier.

Effort de maintenance. La chaîne a elle-même un coût de possession. Le repère le plus visible est le nombre de scripts maintenus (une douzaine sous `scripts/`), les versions épinglées de l'image de compilation à mettre à jour et les jetons d'accès aux services d'analyse et de publication.

5.2.3 Retour des utilisateurs

Aucun recueil formel n’a été mené. Les échanges avec les trois développeurs de l’équipe COM et le Product Owner (chapitre 4) ont orienté les besoins mais ne constituent pas une mesure de satisfaction.

5.2.4 Apports pour l’entreprise

Apport technique. Les développeurs applicatifs disposent d’un compilateur croisé (le SDK) et d’une image de référence publiés dans Artifactory sans reconstruire la distribution, et d’un environnement de compilation unique pour la compilation native et croisée (section 4.4.1). L’inventaire des composants tiers, qui n’existait pas de manière automatisée, est désormais produit à chaque exécution (4 composants identifiés dans l’archive de production).

Apport organisationnel. La qualité devient visible : le tableau de bord Grafana affiche en permanence le niveau de couverture (32,3 % des lignes), les défauts ouverts (3 de priorité 1 et 46 de priorité 2) et la répartition des vulnérabilités. Cette visibilité donne à l’équipe et au Product Owner une base de discussion factuelle. Elle fournit aussi des éléments de preuve pour les obligations de traçabilité des composants évoquées en section 3.2.

5.3 Difficultés rencontrées

Cette section consolide les difficultés rencontrées pendant la mise en œuvre, détaillées dans les sections du chapitre 4 (par exemple section 4.4.1 pour la chaîne applicative), et celles propres à la mesure.

5.3.1 Difficultés techniques

Les difficultés techniques, détaillées au chapitre 4, relèvent de cinq familles : un conteneur imposé en superutilisateur alors que BitBake refuse ce mode (section 4.4.1) ; des chemins qui varient selon l’exécuteur et l’image, pour les artefacts comme pour l’analyse statique (sections 4.4.1 et 4.4.1) ; des services système à démarrer dans un conteneur pour les tests d’intégration (section 4.4.1) ; des étapes susceptibles de bloquer un exécuteur, bornées par `timeout-minutes` ; enfin des codes de sortie et des formats d’outils mal interprétés par la plateforme (section 4.4.4).

5.3.2 Difficultés liées à la mesure

Trois obstacles ont affecté la collecte des mesures.

Une mesure mêlée à la mise au point. Les échecs se concentrent sur les branches où la chaîne était développée : six échecs successifs de `pm77xx-app` sur `feat/runner-ci-collect-metrics` le 13 août (6,3 à 25,9 min), des échecs de `pm77xx-yocto` sur `feat/publish-bsp-images` les 11 et 12 août, trois échecs sur `feat/BE-2338-robot-framework-integration` du

7 au 9 septembre, et trois échecs de **publish-image** à l'étape de construction le 10 septembre (2,8 à 3,1 min). Le taux d'échec brut surestime donc le taux d'échec du produit.

Une chaîne sans historique. Les indicateurs n'étaient pas historisés avant la mise en place de la collecte (section 4.4.3) : il n'existe pas de point de comparaison antérieur.

Des instantanés non rejouables. Les valeurs de couverture, de défauts et de vulnérabilités sont lues dans des interfaces à une date donnée. Seules les durées et les issues sont reconstituables par l'API.

5.4 Résultats

Cette section présente les résultats quantitatifs dans l'ordre reproductibilité, performance, puis qualité et sécurité. Les résultats de performance reposent sur le relevé des exécutions ; ceux de qualité sont des instantanés datés.

5.4.1 Reproductibilité de l'environnement

La reproductibilité repose sur trois dispositifs décrits au chapitre 4 : les versions de tous les outils sont épinglées dans l'image de compilation (tableau 4.11), la même image sert à la compilation native et croisée, et une image n'est promue sous son étiquette définitive qu'après avoir construit l'application complète (section 4.4.1).

Le relevé fournit une preuve indirecte. Sur la période, les exécutions planifiées de **pm77xx-app** sur la branche **main** se situent dans une plage étroite (20,1 à 26,0 min, hors les deux valeurs aberrantes expliquées en section 5.1), et les exécutions de la chaîne de distribution réussies dans une plage de 63,7 à 77,0 min. Cette régularité est compatible avec un environnement stable, sans le démontrer : la durée ne prouve pas l'identité des artefacts.

5.4.2 Performance de la chaîne

Durées de construction

Le tableau 5.3 donne les durées en minutes, toutes issues confondues. Les minimums correspondent à des exécutions annulées ou échouées immédiatement. La médiane de **publish-image** reflète trois échecs sur cinq à l'étape de construction (2,8 à 3,1 min) ; les deux exécutions réussies durent 47,9 et 49,3 min.

Table 5.3 – Durées de construction observées (minutes)

Chaîne	Exécutions	Minimum	Médiane	Maximum
pm77xx-app (matrice complète)	108	0,2	21,8	137,3
pm77xx-yocto (cinq variantes)	51	1,9	68,2	77,0
publish-image	5	2,8	3,1	49,3

La durée médiane de la chaîne applicative (21,8 min) est compatible avec l'exigence ENF4 : le résultat complet (compilation, analyses, tests et couverture) est disponible en moins d'une demi-heure. Les exécutions planifiées montrent une baisse nette au cours de la période : de 24,7 à 26,0 min du 11 au 19 août à 20,1 à 21,4 min du 20 au 31 août. Cette baisse suit les essais de la branche `refactor/pm77xx-app` (18 et 19 août) ; le lien est vraisemblable mais non établi, d'autres modifications ayant eu lieu au même moment (section 5.1).

Durée de la distribution et cache

La distribution est construite à froid à chaque exécution : la médiane est de 68,2 min pour cinq variantes en parallèle, avec une plage de 63,7 à 77,0 min entre les exécutions réussies. Cette faible dispersion confirme l'absence d'effet de cache d'une exécution à l'autre, puisque les répertoires `DL_DIR` et `SSTATE_DIR` sont détruits avec l'exécuteur (section 4.4.2). La comparaison construction à froid et à chaud n'a pas pu être réalisée : l'étude a retenu une solution NFS, qui n'est pas encore mise en œuvre. Cette absence est présentée comme une limite et comme la première perspective (section 5.5).

Effet du parallélisme de la matrice

Sur l'exécution #1316, les quatre cellules durent 19 min 28 s, 5 min 06 s, 5 min 17 s et 4 min 55 s, soit 34 min 46 s si elles étaient exécutées l'une après l'autre. La durée réelle de l'exécution est de 23 min 06 s, soit un gain d'environ 33 %. Ce gain est borné par la cellule `DEBUG/amd64`, qui porte les tests et la couverture et occupe à elle seule 19 min 28 s : ajouter de la parallélisation entre cellules n'y changerait rien, seul un découpage de cette cellule le ferait.

5.4.3 Qualité et sécurité

Couverture de code

Sur l'exécution #1316, la couverture mesurée par `gcovr` est de 32,3 % des lignes (19 308 sur 59 772), 20,9 % des fonctions (1 511 sur 7 216) et 29,6 % des branches (5 559 sur 18 775). Les trois indicateurs sont sous le seuil bas de 50 % configuré dans la synthèse. Ce niveau est connu et rendu visible plutôt que masqué : la mise en place de la mesure est un résultat en soi, puisqu'aucune valeur n'existait auparavant.

Défauts détectés par l'analyse statique

Les vues Coverity restreintes au code de l'équipe comptent 3 défauts de priorité 1 et 46 de priorité 2 à la date de la capture. Ce stock correspond à la dette initiale du projet et justifie la politique de non-blocage décrite en section 4.4.1. Distinguer défauts préexistants et défauts nouvellement introduits suppose un historique que le relevé ne fournit pas.

Vulnérabilités des composants tiers

L'analyse BDBA de l'archive RELEASE/aarch64 identifie 4 composants dont 3 vulnérables (cjson, openldap, hostapd), pour 122 vulnérabilités : 5 critiques, 19 élevées, 86 moyennes et 12 faibles (figure 4.5). L'apport principal est l'existence même de cet inventaire : avant la mise en place, il n'était pas produit de manière automatisée, et chaque exécution le recalculait désormais.

5.5 Discussion

5.5.1 Réponses aux questions d'évaluation

QE1 — Reproductibilité. Réponse partielle. Les dispositifs sont en place (versions épinglées, image unique pour le natif et le croisé, image promue après validation) et les durées sont stables, mais l'identité des artefacts n'a pas été vérifiée.

QE2 — Analyses de sécurité et vitesse. Réponse affirmative sur la vitesse : avec Coverity limité à une cellule de la matrice et BDBA exécuté en aval, la chaîne applicative reste à une médiane de 21,8 min. Sur la détection, la chaîne remonte 3 défauts de priorité 1, 46 de priorité 2 et 122 vulnérabilités de composants, mais l'absence d'historique ne permet pas de montrer une amélioration dans le temps.

QE3 — Pilotage par les indicateurs. Réponse affirmative sur le principe : couverture, défauts, vulnérabilités, taille d'artefact et état des chaînes sont collectés et affichés dans un tableau de bord commun. L'usage de ces indicateurs pour décider n'a pas été mesuré.

QE4 — Maîtrise de l'infrastructure. Réponse partielle. Les jobs légers sont séparés des jobs lourds, mais la distribution est reconstruite à froid à chaque exécution (68,2 min, de l'ordre de 90 % des 30 heures d'exécuteur hebdomadaires estimées). La maîtrise du coût passe par un cache partagé qui reste à mettre en œuvre.

5.5.2 Limites

Les limites de la solution sont les suivantes :

- aucun test n'est exécuté sur la cible matérielle réelle : seule la compilation vise l'architecture aarch64, les tests s'exécutent sur amd64 ;
- la politique de blocage n'est pas appliquée à l'analyse statique, qui signale les défauts sans faire échouer l'exécution ;
- les tests d'intégration Robot Framework sont désactivés par défaut et limités à la cellule DEBUG/amd64 ;
- le cache de la distribution n'est pas partagé ;
- le conteneur de compilation s'exécute en superutilisateur, par contrainte du service d'exécuteurs ;
- la couverture reste faible (32,3 % des lignes) ;

- les artefacts applicatifs portent une version fixe et un horodatage, sans empreinte de commit ;
- aucune mesure de référence antérieure n'existe, et la période d'observation est de cinq semaines.

5.5.3 Menaces à la validité

Trois éléments pourraient invalider les conclusions. D'abord, l'étude porte sur un seul produit et une seule équipe, avec un service d'exécuteurs propre à l'entreprise. Ensuite, plusieurs évolutions de la chaîne ont eu lieu en même temps (refactorisation, collecte de métriques, Robot Framework), de sorte qu'un effet mesuré ne peut être rattaché à une seule cause, comme la baisse de durée de la chaîne applicative autour du 20 août. Enfin, les valeurs de couverture, de défauts et de vulnérabilités sont des instantanés lus dans des interfaces, et l'équipe a pu progresser indépendamment de la chaîne pendant la période.

Chapitre 6

Conclusion et perspectives

6.1 Rappel de la problématique et de la démarche

Ce mémoire part d’une question : comment concevoir et mettre en œuvre une chaîne d’intégration et de livraison continues reproductible, sécurisée et mesurable pour le firmware d’un produit industriel embarqué, sous contraintes de temps de construction longs, d’exigences de cybersécurité et d’infrastructure d’exécution éphémère. La démarche a consisté à reprendre une chaîne existante plutôt qu’à en créer une : analyse de l’existant et recueil des besoins auprès de l’équipe COM, formalisation d’exigences fonctionnelles et non fonctionnelles, conception, mise en œuvre sur trois chaînes (application, distribution Linux embarquée, image de compilation), puis évaluation sur cinq semaines d’exécutions réelles.

6.2 Principaux résultats

Le tableau 6.1 reprend les cinq contributions annoncées en section 1.4 avec les résultats du chapitre 5.

Table 6.1 – Contributions et résultats

Contribution		Résultat
Conteneurisation de l'environnement de compilation	de	Une image unique, aux versions d'outils épinglées et intégrant le SDK, sert à la compilation native et croisée ; elle n'est publiée qu'après avoir construit l'application complète.
Deux chaînes consolidées		Application : durée médiane de 21,8 min ; la matrice 2×2 ramène 34 min 46 s de calcul cumulé à 23 min 06 s observées (environ -33%). Distribution : cinq variantes d'image et SDK en 68,2 min médian.
Analyses de sécurité		Coverity restitue 3 défauts de priorité 1 et 46 de priorité 2 ; BDBA identifie 4 composants et 122 vulnérabilités, dans la synthèse de chaque exécution.
Contraintes des exécuteurs éphémères		Contraintes documentées (droits, chemins, absence de cache) ; la distribution est reconstruite à froid à chaque exécution ; étude NFS et snapshot réalisée, solution NFS retenue mais non encore mise en œuvre.
Observabilité		Métriques envoyées à VictoriaMetrics et tableau de bord Grafana : couverture (32,3 % des lignes), taille d'artefact (78,0 Mo), état des chaînes.

Ces résultats répondent de manière inaboutie à deux des quatre questions d'évaluation : la reproductibilité repose sur des dispositifs en place mais l'identité des artefacts n'a pas été vérifiée, et la maîtrise du coût d'exécution reste limitée par l'absence de cache partagé.

6.3 Apports personnels

6.3.1 Montée en compétences techniques

Je maîtrisais déjà les bases de l'intégration continue et du scripting. Ces missions m'ont fait découvrir ou approfondir des outils que je ne maîtrisais pas à mon arrivée : construction d'images de conteneurs avec BuildKit, génération d'une distribution Linux embarquée avec Yocto et kas, intégration de Coverity et de BDBA, tests d'intégration avec Robot Framework, puis collecte et restitution de métriques avec VictoriaMetrics et Grafana.

En parallèle, j'ai exploré l'apport de l'intelligence artificielle dans la chaîne CI/CD en développant un agent qui m'aide à corriger les erreurs des exécutions. Une tâche de correction qui me prenait environ une heure ne me demande plus qu'environ cinq minutes avec l'agent. Ce chiffre est une estimation personnelle, non mesurée systématiquement ; il indique un potentiel plus qu'un résultat établi.

6.3.2 Intégration dans une organisation industrielle

Au-delà de l'aspect technique, ce stage a été ma première expérience du fonctionnement d'un grand groupe industriel et international. Le bilan que j'en tire est que les compétences relationnelles ont pesé plus lourd que les compétences techniques. Un outil se prend en main avec de la documentation ; en revanche, comprendre une chaîne héritée, distinguer ce qui est imposé de ce qui se décide, et rendre les résultats utiles à ceux qui s'en servent dépendent surtout des échanges avec les autres.

L'équipe COM travaille en méthode agile, et trois rituels ont structuré ces échanges. La planification de sprint m'a permis de cadrer mes tâches et de négocier leur priorité avec le Product Owner ; la réunion quotidienne, de signaler tôt un blocage et de demander de l'aide plutôt que de le contourner seul ; la démonstration de fin de sprint, de présenter mes avancées (tableau de bord, synthèses, analyse BDBA) et d'en recueillir les retours. Mon tuteur, coordinateur DevSecOps, m'a aidé à distinguer ce qui relevait de contraintes de l'entreprise de ce que je pouvais décider. J'ai aussi appris à présenter honnêtement les limites de mon travail, comme la politique de non-blocage de l'analyse statique ou le faible niveau de couverture, plutôt que de les masquer.

6.3.3 Projet professionnel

Ce travail confirme mon souhait de poursuivre dans le domaine du DevSecOps. Il relie trois sujets qui m'intéressent : l'automatisation des chaînes de construction, la sécurité de la chaîne d'approvisionnement logicielle (analyse de composition, nomenclature logicielle, signature des artefacts) et la qualité rendue mesurable. Les perspectives ci-dessous décrivent le travail que je souhaiterais y mener.

6.4 Perspectives

Les perspectives découlent des limites du chapitre 5.

6.4.1 Court terme

- Blocage de la chaîne sur les défauts Coverity nouvellement introduits, la comparaison de vues de `pyCovXtract.py` permettant de les isoler de la dette initiale.
- Activation par défaut des tests d'intégration Robot Framework.
- Ajout de l'empreinte du commit et de l'étiquette de l'image de compilation au nom des artefacts publiés.
- Passage du jeton BDBA par l'environnement et non par la ligne de commande.
- Exécution de la chaîne à chaque proposition de modification et non uniquement selon une planification quotidienne.
- Suppression de l'exécution en superutilisateur dans le conteneur de compilation.

6.4.2 Moyen terme

- Mise en œuvre du cache partagé NFS retenu par l'étude, afin de réduire une construction qui repart aujourd'hui à froid à chaque exécution.
- Intégration du banc de test matériel dans la chaîne, afin d'exécuter les tests sur cible réelle et non uniquement en compilation native.
- Génération et publication d'une nomenclature logicielle normalisée associée à chaque livrable.
- Signature et attestation de provenance des artefacts produits.

6.4.3 Long terme

- Extension de la chaîne aux autres produits de la gamme, notamment à l'équipe MET, par factorisation des workflows réutilisables.
- Automatisation de la constitution du dossier de preuves attendu par le cadre normatif.
- Intégration de l'agent d'aide à la correction dans la chaîne, après mesure de son apport sur un plus grand nombre de cas.

6.5 Conclusion

La question était de savoir comment rendre la chaîne d'intégration et de livraison d'un firmware embarqué reproductible, sécurisée et mesurable. Ce travail y apporte une réponse opérationnelle : la chaîne applicative rend un résultat complet en 21,8 minutes en médiane, les analyses de sécurité sont intégrées à chaque exécution avec restitution au développeur, et les indicateurs de qualité sont historisés et affichés dans un tableau de bord commun. Cette réponse reste partielle sur deux points identifiés : la distribution Linux est encore reconstruite à froid en 68,2 minutes et l'identité des artefacts d'une construction à l'autre n'a pas été vérifiée. Pour l'équipe COM comme pour Schneider Electric, l'apport est une chaîne en service quotidien dont l'état se lit sans outil supplémentaire ; la prochaine étape est le cache partagé de la distribution.

Bibliographie

- [1] M. BELLER, R. BHOLANATH, S. MCINTOSH et A. ZAIDMAN. « Analyzing the State of Static Analysis : A Large-Scale Evaluation in Open Source Software ». In : *Proceedings of the 23rd IEEE International Conference on Software Analysis, Evolution, and Reengineering (SANER)*. 2016, p. 470-481.
- [2] P. M. DUVAL, S. MATYAS et A. GLOVER. *Continuous Integration : Improving Software Quality and Reducing Risk*. Addison-Wesley, 2007. ISBN : 978-0321336385.
- [3] N. FORSGREN, J. HUMBLE et G. KIM. *Accelerate : The Science of Lean Software and DevOps : Building and Scaling High Performing Technology Organizations*. IT Revolution Press, 2018.
- [4] M. FOWLER. *Continuous Integration*. 2006. URL : <https://martinfowler.com/articles/continuousIntegration.html> (visit   le 17/09/2026).
- [5] M. HILTON, T. TUNNELL, K. HUANG, D. MARINOV et D. DIG. « Usage, Costs, and Benefits of Continuous Integration in Open-Source Projects ». In : *Proceedings of the 31st IEEE/ACM International Conference on Automated Software Engineering (ASE)*. 2016, p. 426-437.
- [6] J. HUMBLE et D. FARLEY. *Continuous Delivery : Reliable Software Releases through Build, Test, and Deployment Automation*. Addison-Wesley, 2010. ISBN : 978-0321601919.
- [7] *IEC 62443-4-1 :2018 – Security for industrial automation and control systems – Part 4-1 : Secure product development lifecycle requirements*. International Electrotechnical Commission. 2018.
- [8] *ISO/IEC 5962 :2021 – Information technology – SPDX® Specification V2.2.1*. International Organization for Standardization. 2021.
- [9] C. LAMB et S. ZACCHIROLI. « Reproducible Builds : Increasing the Integrity of Software Supply Chains ». In : *IEEE Software* 39.2 (2022), p. 62-70.
- [10] L. E. LWAKATARE, T. KILAMO, T. KARVONEN, T. SAUVOLA, V. HEIKKIL  , J. M  NCH, M. OIVO et C. LASSENIUS. « DevOps in Practice : A Multiple Case Study of Five Companies ». In : *Information and Software Technology* 114 (2019), p. 217-230.
- [11] T. M  RTENSSON, D. ST  HL et J. BOSCH. « Continuous Integration Applied to Software-Intensive Embedded Systems – Problems and Experiences ». In : *Product-Focused Software Process Improvement (PROFES)*. T. 10027. Lecture Notes in Computer Science. Springer, 2016, p. 448-457.

- [12] NATIONAL TELECOMMUNICATIONS AND INFORMATION ADMINISTRATION. *The Minimum Elements For a Software Bill of Materials (SBOM)*. Rapp. tech. U.S. Department of Commerce, 2021. URL : <https://www.ntia.gov/report/2021/minimum-elements-software-bill-materials-sbom>.
- [13] M. OHM, H. PLATE, A. SYKOSCH et M. MEIER. « Backstabber's Knife Collection : A Review of Open Source Software Supply Chain Attacks ». In : *Detection of Intrusions and Malware, and Vulnerability Assessment (DIMVA)*. Springer, 2020, p. 23-43.
- [14] OWASP FOUNDATION. *OWASP Top 10 – The Ten Most Critical Web Application Security Risks*. URL : <https://owasp.org/www-project-top-ten/> (visité le 20/09/2026).
- [15] PARLEMENT EUROPÉEN ET CONSEIL DE L'UNION EUROPÉENNE. *Règlement (UE) 2024/2847 du 23 octobre 2024 concernant des exigences horizontales de cybersécurité pour les produits comportant des éléments numériques (Cyber Resilience Act)*. Journal officiel de l'Union européenne. 2024. URL : <https://eur-lex.europa.eu/eli/reg/2024/2847/oj>.
- [16] M. SHAHIN, M. A. BABAR et L. ZHU. « Continuous Integration, Delivery and Deployment : A Systematic Review on Approaches, Tools, Challenges and Practices ». In : *IEEE Access* 5 (2017), p. 3909-3943.
- [17] D. STÅHL et J. BOSCH. « Modeling Continuous Integration Practice Differences in Industry Software Development ». In : *Journal of Systems and Software* 87 (2014), p. 48-59.
- [18] B. VASILESCU, Y. YU, H. WANG, P. DEVANBU et V. FILKOV. « Quality and Productivity Outcomes Relating to Continuous Integration in GitHub ». In : *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering (ESEC/FSE)*. 2015, p. 805-816.
- [19] F. ZAMPETTI, S. SCALABRINO, R. OLIVETO, G. CANFORA et M. DI PENTA. « How Open Source Projects Use Static Code Analysis Tools in Continuous Integration Pipelines ». In : *Proceedings of the 14th International Conference on Mining Software Repositories (MSR)*. 2017, p. 334-344.

Données de mesure

.1 Relevé des exécutions

Le relevé complet (200 exécutions, dont 36 de la chaîne hors périmètre `evo-bench-image`) est produit par `scripts/collecte-mesures.sh` et conservé dans `mesures/executions.csv`. Le tableau 2 en donne la synthèse par chaîne ; le tableau 3 en présente un extrait.

Table 2 – Synthèse des exécutions par chaîne (11 août – 17 septembre 2026)

Chaîne	Ex.	Succès	Échecs	Taux (%)	Min	Méd.	Max
pm77xx-app	108	86	19	79,6	0,2	21,8	137,3
pm77xx-yocto	51	41	9	80,4	1,9	68,2	77,0
publish-image	5	2	3	40,0	2,8	3,1	49,3

Table 3 – Extrait du relevé des exécutions

Id.	Date	Chaîne	Issue	Durée (min)
13128034	2026-09-17	pm77xx-app (planifiée)	succès	31,7
13126425	2026-09-16	pm77xx-yocto (planifiée)	succès	69,6
13095487	2026-09-16	pm77xx-app (planifiée)	succès	21,0
12976540	2026-09-10	publish-image	succès	47,9
12973371	2026-09-10	publish-image	échec	3,1
12911060	2026-09-08	pm77xx-yocto (planifiée)	échec	68,1
12425229	2026-08-21	pm77xx-app (planifiée)	succès	137,3
12243079	2026-08-13	pm77xx-app (manuelle)	échec	6,7
12145741	2026-08-11	pm77xx-app (planifiée)	échec	12,7

.2 Mesures de couverture

Exécution #1316 de `pm77xx-app` (DEBUG/amd64) : lignes 32,3 % (19 308 / 59 772), fonctions 20,9 % (1 511 / 7 216), branches 29,6 % (5 559 / 18 775). Seuils de la synthèse : 50 % (bas) et 75 % (haut).

.3 Relevé des défauts d'analyse statique

Vues Coverity du projet, à la date de la capture : 3 défauts de priorité 1 (vue P1) et 46 de priorité 2 (vue P2).

.4 Relevé des vulnérabilités de composants

Archive `pm77xx-app RELEASE/aarch64`, produit `pm77xx-app` version 1.0.0 : 4 composants identifiés dont 3 vulnérables (`cjson`, `openldap`, `hostapd`) ; 122 vulnérabilités, soit 5 critiques, 19 élevées, 86 moyennes et 12 faibles.